

Efficient and Post-Quantum Conjunctive Dynamic SSE with Strong Privacy Guarantees

Bibhas Chandra Das^{1,2}, Nilanjan Datta^{1,3}, Avijit Dutta^{1,3}, Avishek Majumder⁴, Debdeep Mukhopadhyay⁵, Sikhar Patranabis⁶, Subhabrata Samajder^{1,3}, and Laltu Sardar⁷

¹ Institute for Advancing Intelligence, TCG CREST, Kolkata, India
{bibhaschandra.das, nilanjan.datta, avijit.dutta, subhabrata.samajder}@tcgcrest.org

² Chennai Mathematical Institute, Chennai, India

³ Academy of Scientific and Innovative Research, Ghaziabad, India

⁴ Krea University, India
avishek.majumder@krea.edu.in

⁵ Indian Institute of Technology, Kharagpur, India
debdeep@cse.iitkgp.ac.in

⁶ IBM Research, Bengaluru, India
sikhar.patranabis@ibm.com

⁷ Indian Institute of Science Education and Research, Thiruvananthapuram, India
laltu.sardar@iisertvm.ac.in

Abstract. Designing dynamic searchable symmetric encryption (DSSE) supporting conjunctive keyword queries over encrypted document collections is an important research area. Unfortunately, state-of-the-art conjunctive DSSE schemes such as ODXT (Patranabis et al., NDSS '21), and SDSSE-CQ and its variants (Zuo et al., PoPETS '25) either fail to achieve the desired levels of security (in particular, forward and/or backward privacy), or incur prohibitively large communication requirements and client-side computational overheads, which is undesirable for practical applications. In addition, all known conjunctive DSSE schemes that are both forward and backward private are quantum-broken due to their inherent reliance on discrete log-hard, prime-order cyclic groups.

In this paper, we address the open question of designing practically efficient, low-leakage, forward and backward private conjunctive DSSE with small communication requirements and small client-side overheads. Towards this goal, we introduce the first systematic and rigorously formal notions of backward privacy for conjunctive queries that naturally extend the corresponding notions of backward privacy for single-keyword DSSE. Next, as our main contribution, we propose a new framework for conjunctive DSSE called fp-GA-ODXT that improves upon both ODXT and SDSSE-CQ-variants by achieving full-fledged forward privacy and strong backward privacy guarantees, while incurring small client-side computation and low communication overheads. We then demonstrate two instantiations of our fp-GA-ODXT framework:

- A concretely efficient, classically secure instance based on discrete log-hard groups and an RSA-based trapdoor permutation, which we

implement and benchmark. Our experiments demonstrate that we achieve smaller communication overheads and client-side computation for both updates and conjunctive searches as compared to both ODXT and SDSSE-CQ-variants. These come at the cost of a mild increase in server computation (which we view as an acceptable trade-off in practice).

- A post-quantum instantiation based on lattices and isogenies of supersingular elliptic curves, thus solving the longstanding open question of designing asymptotically efficient, forward and backward private conjunctive DSSE schemes with post-quantum security.

Keywords: Dynamic SSE, Forward Privacy, Backward Privacy, Post Quantum Security.

1 Introduction

Searchable Symmetric Encryption (SSE) allows a client to outsource a symmetrically encrypted document collection (where each document/file in the collection is tagged with a set of keywords) to an (untrusted) server, while retaining the ability to directly execute keyword queries [38,16]. Other cryptographic primitives supporting encrypted search, such as fully homomorphic encryption (FHE) [7] and oblivious RAM [39,22], minimize information leakage to the server, but incur high computational/communication overheads in the process. On the other hand, SSE achieves a reasonable trade-off between security and efficiency by minimizing the computational and communication overheads, while adhering to a rigorously formalized leakage function. Security analysis of SSE typically involves two steps: (i) a simulation-based proof that establishes that an SSE scheme leaks no information to the server beyond what is captured by the leakage function, and (ii) a cryptanalysis-based argument to establish that the leakage function is benign, i.e., does not leak sensitive information about the client’s data/queries.

Static SSE. Initial studies of SSE were restricted to the very simple functionality of *single keyword* search over *static* databases (e.g., [38,21,16]). A more recent line of works has extended the study of SSE over static databases to richer functionalities beyond single keyword search, including conjunctive, disjunctive, and more general Boolean queries [10,19,25,27]. However, to be truly practical, an SSE scheme should ideally support both keyword searches and *dynamic updates* to the database, simultaneously.

Dynamic SSE. The study of SSE for dynamic databases (abbreviated as DSSE throughout) was initiated in [14,26,9]. However, these schemes were found to be vulnerable to data/query recovery attacks [8,46]. Subsequently, a line of works [5,6] introduced two security notions for DSSE, which have been adopted widely in several subsequent DSSE schemes [15,41,13,48,40]:

- *Forward privacy*: informally ensures that the update operation cannot be linked to any previous search operations.
- *Backward privacy*: informally ensures that during identical search queries, the server should not be able to learn about documents/files that were previously added and later deleted.

The authors of [6] further described three categories of backward privacy that leak progressively more information to the server about a queried keyword w : (i) **Type-I** backward privacy leaks the documents currently matching w , the time of their insertion, and the total number of updates on w ; (ii) **Type-II** backward privacy leaks, in addition to **Type-I**, when all updates on w happened without leaking their specific content; (iii) **Type-III** backward privacy, in addition to **Type-II** leakages, leaks the information about which deletion cancels which insertion. By definition, **Type-I** is the strongest, and **Type-III** is the weakest notion of backward privacy.

Several works [6,15,41,13,48,40] have subsequently proposed DSSE schemes supporting single keyword queries while achieving both forward privacy and (any one of the aforementioned notions of) backward privacy. While many of these schemes are extremely efficient, they are unfortunately too limited in terms of query expressiveness. Many practical applications require the client to query the *conjunction* of two or more keywords, i.e., identify the documents that contain *all* keywords in a given set. This motivates the need for DSSE schemes satisfying suitable notions of forward and backward privacy while supporting conjunctive keyword queries, which is the focus of this work.

Conjunctive DSSE. A conjunctive search query $q = w_1 \wedge w_2 \wedge \dots \wedge w_n$ requires the server to return documents containing all the keywords $w_1, w_2, \dots, w_n$. Note that one can achieve conjunctive DSSE from single-keyword DSSE in a black-box manner by simply invoking the single-keyword DSSE to query each w_i , and then computing the intersection locally at the client. Besides incurring additional undesirable leakage (e.g., the frequency of each keyword), this approach incurs computational and communication overheads that scale with the frequency of the *most frequent keyword*. As noted in [10], this is highly undesirable in practice. Certain early works on conjunctive DSSE [23,45,44] proposed more sophisticated schemes for conjunctive SSE, but these schemes only achieve forward privacy, and not backward privacy.

Conjunctive DSSE from the OXT Template. In the setting of static SSE, a template for designing practically efficient conjunctive SSE was introduced in the widely studied OXT protocol of [10]. In particular, OXT supports highly efficient, single-round, low-leakage conjunctive keyword searches, where the computational and communication complexity only scales with the frequency of the *least* frequent keyword. The first work to translate the template of OXT into the setting of dynamic databases was ODXT [33]. ODXT supports efficient, single-round conjunctive keyword queries with the same asymptotic computational and communication overheads as OXT, while also supporting efficient,

non-interactive updates. Unfortunately, as noted in [47], ODXT *does not* achieve the desired level of forward privacy.

Alternatives to ODXT. Concurrent to [33], the authors of [49] proposed FBDSSC-CQ and SFBDSSE-CQ – two conjunctive DSSE schemes achieving both forward privacy and (a weaker notion of) backward privacy. However, both constructions incur computational and communication overheads that scale with the frequency of the *most frequent keyword* (same as the naïve solution). In practice, for moderately sized databases, both constructions incur query communication in the range of gigabytes, which is prohibitively large. As a result, these schemes do not scale to large databases in practice.

A follow-up work [47] proposed two new conjunctive DSSE schemes based on the same OXT framework, called SDSSE-CQ and SDSSE-CQ-S. Both constructions use Aura [40], a forward and backward private DSSE scheme for single keyword search, as their underlying protocol. While these schemes fix the forward privacy issues of ODXT [33], they incur additional search leakage, resulting in significantly weaker backward privacy guarantees. One can easily observe that for a conjunction of n keywords, the search protocol of both these schemes additionally invokes n instances of the search protocol of the underlying single keyword SSE. Furthermore, both of these schemes incur substantial communication and computation overhead on the client side. This is highly undesirable, in particular, in many practical applications of SSE, where the client could be resource-constrained. For such applications, high client computation represents a major bottleneck for practical scalability. Alongside this, in a very recent work [18], Das has identified an inconsistency in the stronger backward privacy claim of SDSSE-CQ-S and has shown that both SDSSE-CQ and SDSSE-CQ-S guarantee a very weak level of backward privacy.

A possible approach towards fixing this state of affairs would be to design a conjunctive DSSE scheme that achieves both forward and backward privacy *and* reduces the computational and communication overheads for the client, while potentially incurring larger server-side computation. Since the server (e.g., a cloud service provider) is typically equipped with stronger computational capabilities, this would be a much more reasonable tradeoff. To the best of our knowledge, no existing conjunctive DSSE scheme achieves this property. This motivates us to ask the following question:

Can we design efficient, low-leakage, forward and backward private conjunctive DSSE with small communication and client-side overheads?

Ad-hoc nature of Backward Privacy Definitions. A closer look reveals another issue with existing forward and backward private conjunctive DSSE schemes [33,49,47], namely, all of these works introduce ad-hoc notions of backward privacy that are only satisfied by their specific scheme. Such definitions are fundamentally incompatible with the systematic categorizations of backward privacy leakage introduced in [6] for single-keyword DSSE and used consistently in several subsequent works [15,41,13,48,40]. The lack of formal, uniformly applicable

notions of backward privacy for conjunctive DSSE not only makes it difficult to compare the existing schemes [33,49,47] in terms of their leakage, but also fails to provide future efforts to design conjunctive DSSE with a reasonable baseline/framework for evaluating their leakage functions.

Lack of Post-Quantum Conjunctive DSSE. Finally, we note that all of the conjunctive DSSE schemes today with forward and backward privacy [33,49,47] fundamentally rely on the hardness of the discrete logarithm problem over cyclic, prime-order groups (this is, in fact, inherited from the original design of OXT in [10]). As a result, they would be fundamentally broken by a quantum computer capable of running Shor’s algorithm. We note here that a recent line of works [42,43] have proposed post-quantum conjunctive *static* SSE schemes from lattice-based assumptions. However, these schemes have no dynamic counterparts today. This leads to the following question:

Can we design a post-quantum conjunctive DSSE scheme?

1.1 Our Contributions

We answer both of the above questions in the affirmative. Our contributions are summarized below.

Systematizing Backward Privacy of Conjunctive DSSE. As a first contribution, we propose a new formal framework for systematizing backward privacy of conjunctive DSSE. We extend and formalize the existing backward privacy notions - Type-I, Type-II, and Type-III - originally designed for single-keyword DSSE, adapting them for conjunctive queries. We categorize backward privacy for conjunctive DSSE into three classes, C1, C2, and C3, which naturally build upon the existing backward privacy notions while capturing the possible information leakage in conjunctive searches. Additionally, we introduced a refined notion of backward privacy, dubbed $C2^-$, which is particularly relevant for OXT-based DSSE protocols, and offers stronger backward privacy guarantees than C2. To the best of our knowledge, this represents the first attempt to systematically formalize backward privacy notions for DSSE schemes that support conjunctive queries. In particular, by unifying previous definitions, our model provides a clearer understanding of the security of existing schemes. Notably, our analysis confirms that ODXT achieves the strongest backward privacy among all known conjunctive DSSE schemes.

fp-GA-ODXT. As our main contribution, we design fp-GA-ODXT – a variant of ODXT that achieves full-fledged forward privacy while retaining strong backward privacy, low client-side computation, and small communication overhead. To this end, we first introduce an intermediate, cryptographic group action [2] based generalization of ODXT, that we call GA-ODXT. More specifically, unlike ODXT which relies explicitly on the classical hardness of discrete log over cyclic, prime-order groups, GA-ODXT relies on the hardness of the group action-based

Table 1. Comparison of state-of-the-art conjunctive DSSE schemes that achieve both forward and backward privacy. Here, W and D denote the number of keywords and file identifiers in the database, respectively. For a conjunctive search query of the form $(w_1 \wedge w_2 \wedge \dots \wedge w_n)$, where w_1 is the least frequent keyword, c_{w_i} for each $i \in [n]$ denotes the number of update operations involving the keyword w_i .

Construction	Forward Backward		Communication		Client Computation		Storage
	Privacy	Privacy	Update	Search	Update	Search	
ODXT[33]	$\times$	$C2^-$	$O(1)$	$O(n \cdot c_{w_1})$	$O(1)$	$O(1)$	$O(W)$
FBDSSE-CQ[49]	$\checkmark$	$C2$	$O(D)$	$O(n + D)$	$O(D)$	$O(D \cdot \sum_{i=1}^n c_{w_i})$	$O(W)$
SFBDSSSE-CQ[49]	$\checkmark$	$C2$	$O(D)$	$O(n + D)$	$O(s \cdot D)$	$O(s \cdot D)$	$O(1)$
SDSSE-CQ[47]	$\checkmark$	$C2-II$	$O(1)$	$O(n \cdot c_{w_1})$	$O(1)$	$O(n \cdot c_{w_1})$	$O(W)$
SDSSE-CQ-S[47]	$\checkmark$	$C2-I$	$O(1)$	$O(n \cdot c_{w_1} + \sum_{i=2}^n c_{w_i})$	$O(1)$	$O(n \cdot c_{w_1} + \sum_{i=2}^n c_{w_i})$	$O(W)$
fp-GA-ODXT [Sec.4.3]	$\checkmark$	$C2^-$	$O(1)$	$O(n + c_{w_1})$	$O(1)$	$O(n + c_{w_1})$	$O(W)$

analogue of the discrete log hardness assumption. However, similar to ODXT, GA-ODXT also does not achieve full-fledged forward privacy. Nonetheless, we leverage this generic abstraction to identify and patch the fundamental source of this issue with forward privacy to propose **fp-GA-ODXT**. Beyond fixing the forward privacy issues with ODXT, another novel design feature of **fp-GA-ODXT** is its usage of *backward-chaining* to (asymptotically) retain the same overall communication overhead, the same client-side overhead during update, and a smaller client-side overhead during search compared to ODXT.

We summarize and compare the security and efficiency of state-of-the-art conjunctive DSSE schemes (that achieve both forward and backward privacy) with **fp-GA-ODXT** in Table 1. We note that, asymptotically, **fp-GA-ODXT** provides the highest level of privacy guarantees (forward privacy and strong $C2^-$ backward privacy) among all efficient conjunctive DSSE schemes and also achieves small client-side computation and low communication overhead. In particular, **fp-GA-ODXT** avoids the pitfalls of prior approaches [49,47] towards fixing the forward privacy issues of ODXT, which result in substantially larger client-side overheads and communication requirements, as well as weaker backward privacy guarantees. As a tradeoff, **fp-GA-ODXT** incurs mildly larger computational overheads at the server. As explained earlier, in most practical realizations of SSE, the client is expected to be resource-constrained, while the server is typically equipped with stronger compute capabilities (which is the fundamental motivation for outsourcing data in the first place). Hence, we view this as a much more acceptable tradeoff in practice.

Concretely Efficient Instantiation of fp-GA-ODXT. We present a classically secure and concretely efficient instantiation of **fp-GA-ODXT** based on a combination of discrete log-hard prime order groups (same as ODXT) and RSA-based trapdoor permutations (for backward-chaining). We implement and benchmark this instantiation, and compare it with both ODXT and SDSSE-CQ/SDSSE-CQ-S [47]. Our results show that: (i) our average update time and communication overheads are similar to ODXT and $1.4\times$ smaller than SDSSE-CQ/SDSSE-CQ-S, (ii) as the number of *xterm* in the conjunction increases, the search communication cost of our scheme remains constant and minimal, whereas it increases

linearly for both ODXT and SDSSE-CQ/SDSSE-CQ-S, and finally (iii) our average client-side search time is about $1.2\times$ smaller than ODXT, $1.6\times$ smaller than SDSSE-CQ, and $3\times$ smaller than SDSSE-CQ-S. As a tradeoff, from the construction presented in Section 4.3, it is clear that our scheme has an increased server computation overhead. As explained earlier, we view this as a reasonable trade-off, especially in practical scenarios where the SSE client is significantly resource-constrained.

Post-quantum Instantiation of fp-GA-ODXT. We next present a post-quantum instantiation of fp-GA-ODXT based on the combined (worst-case) hardness of lattice problems *and* the hardness of computing isogeny maps between supersingular elliptic curves. Concretely, we use known instantiations of effective group actions from well-studied isogeny-based hardness assumptions (in particular, assumptions over CSIDH [12], CSI-FiSh [3], Scallop [20], and PEGASIS [17]) to instantiate the GA-ODXT component of fp-GA-ODXT. A core technical novelty of this construction is that we replace the use of the RSA-based trapdoor permutation for backward-chaining in our classical instantiation with an *injective trapdoor function* based on the hardness of the short integer solutions (SIS) problem [1]. This yields the *first* (to the best of our knowledge) post-quantum candidate for forward and backward private conjunctive DSSE with desirable asymptotic efficiency guarantees and strong levels of security.

We remark that the post-quantum instantiation is likely to be less efficient than the quantum-broken counterpart (this is a natural and somewhat inherent tradeoff that also applies to other cryptographic primitives being standardized, such as KEMs/signatures), and a fair comparison would require one to also benchmark the natural CSIDH-based analogues of ODXT [33] and SDSSE-CQ/SDSSE-CQ-S [47], for which there are also no implementations at the moment. We leave achieving an optimized implementation of the post-quantum solution as an important but independent future work.

2 Preliminaries

Notations. For any natural number $n \in \mathbb{N}$, we write $[n]$ to denote the set $\{1, \dots, n\}$. Similarly, for $i, j \in \mathbb{N}$ with $i < j$, we write $[i, j]$ to denote the set $\{i, i+1, \dots, j\}$. For a random variable X , we write $x \xleftarrow{\$} \mathcal{X}$ to denote that x is sampled uniformly at random from $\mathcal{X}$. The output x of a deterministic algorithm $\mathcal{A}$ is denoted by $x \leftarrow \mathcal{A}$ and the output x of a randomized algorithm $\mathcal{A}^{\$}$ is denoted by $x \xleftarrow{\#} \mathcal{A}^{\$}$. For two variables x, y , we write $x \leftarrow y$ to denote the assignment of the value in y to the variable x . We refer to $\lambda \in \mathbb{N}$ as the security parameter and denote by $\text{poly}(\lambda)$ and $\text{negl}(\lambda)$ as a generic polynomial function and negligible function in λ , respectively.

Databases. We write $\mathcal{W}$ to denote a set of distinct keywords, which we call a *dictionary*. Let $\mathcal{F}$ be a collection of files such that each file $f \in \mathcal{F}$ is associated with an identifier $id \in \mathcal{ID}$, and contains keywords from a set $\mathcal{W}$. A database

DB refers to the set of all keyword-identifier pairs across the files in $\mathcal{F}$. More formally, we have $\text{DB} = \{(w, id) : w \in \mathcal{W}, id \in \mathcal{ID}\}$ such that a given pair $(w, id) \in \text{DB}$ if and only if the keyword w is in a file $f \in \mathcal{F}$ with the identifier id . For a given keyword $w \in \mathcal{W}$, $\text{DB}(w)$ denotes the set of all file identifiers containing w as a keyword. W denotes the number of keywords present in the database DB, i.e.,

$$W = |\{w : (w, id) \in \text{DB for some } id \in \mathcal{ID}\}|$$

Similarly, D denotes the total number of files in the database DB, i.e.

$$D = |\{id : (w, id) \in \text{DB for some } w \in \mathcal{W}\}|$$

Clearly, $W \leq |\mathcal{W}|$ and $D \leq |\mathcal{ID}|$.

Conjunctive Queries. We represent a conjunctive search query over n distinct keywords as $q = w_1 \wedge w_2 \wedge \dots \wedge w_n$ and define the set $\text{DB}(q) = \cap_{i=1}^n \text{DB}(w_i)$ to be the result of the conjunctive search query. We typically assume (without loss of generality) that the keyword w_1 is the least frequently updated keyword in the conjunction q . We refer to w_1 as the *s-term*, and all the other keywords in the conjunction as *x-terms*.

2.1 Cryptographic Primitives

Pseudo-Random Function: Let $\text{Func}(\{0, 1\}^n)$ be the set of all functions from $\{0, 1\}^n$ to $\{0, 1\}^n$ and $F : \{0, 1\}^k \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a family of keyed functions from $\{0, 1\}^n$ to $\{0, 1\}^n$. The PRF advantage $\text{Adv}_{F, \mathcal{A}}^{\text{PRF}}(\lambda)$ of F with respect to a distinguisher $\mathcal{A}$ is defined as the absolute difference between the probability that $\mathcal{A}$ interacting with $F_K = F(K, \cdot)$ for a randomly sampled secret key K outputs 1, and the probability that $\mathcal{A}$ interacting with a function R chosen uniformly at random from $\text{Func}(\{0, 1\}^n)$ outputs 1. That is,

$$\text{Adv}_{F, \mathcal{A}}^{\text{PRF}}(\lambda) \triangleq \left| \Pr_K[\mathcal{A}^{F_K} = 1] - \Pr_R[\mathcal{A}^R = 1] \right|.$$

We say that F is a PRF if the maximum pseudorandom function advantage of F is $\text{negl}(\lambda)$ where the maximum is taken over all distinguishers $\mathcal{A}$ that make polynomial many queries to its oracle.

Hash Function: A hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ is a function that takes a message of arbitrary length and produces a fixed-length string. In this paper, we assume that a hash function behaves like a random oracle.

2.2 Cryptographic Group Actions

We recall the formal definition of cryptographic group actions from [2].

Group Action. Let G be a group and U be a set. Then G is said to *act on* U if there is a function $\star : G \times U \rightarrow U$ that satisfies the following two properties:

1. Identity: For any $u \in U$, we have $e \star u = u$, where e is the identity element of G .
2. Compatibility: $(gh) \star u = g \star (h \star u)$ for any $g, h \in G$ and any $u \in U$.

We use the notation $(G, U, \star)$ to denote such a group action throughout. We consider group actions that satisfy one or more of the following properties:

1. *Abelian:* The group G is abelian.
2. *Transitive:* For every $u_1, u_2 \in U$, there exists a group element $g \in G$ such that $u_2 = g \star u_1$. For such a transitive group action, the set U is called a *homogeneous space* for G .
3. *Faithful:* For each group element $g \in G$, either g is the identity element of G or there exists an element $u \in U$ such that $u \neq g \star u$.
4. *Free:* For each group element $g \in G$, g is the identity element of G if and only if there exists some element $u \in U$ such that $u = g \star u$.
5. *Regular:* Both free and transitive, i.e., the map $f_u : g \mapsto g \star u$ defines a bijection between G and U , for any $u \in U$. Consequently, if G (or U) is finite, then we must have $|G| = |U|$.

Effective Group Action (EGA). An abelian and regular group action $(G, U, \star)$ is *effective* if the following properties are satisfied:

1. The group G is finite and there exist probabilistic polynomial time (PPT) algorithms for:
 - (a) *Membership testing*, i.e., to decide if a given bit string represents a valid group element in G .
 - (b) *Equality testing*, i.e., to decide if two bit strings represent the same group element in G .
 - (c) *Sampling*, i.e., to sample an element g according to some distribution on G . In this paper, we consider distributions that are (statistically close to) uniform.
 - (d) *Operation*, i.e., to compute gh for any $g, h \in G$.
 - (e) *Inversion*, i.e., to compute g^{-1} for any $g \in G$.
2. The set U is finite, and there exist efficient algorithms for:
 - (a) *Membership testing*, i.e., to decide if a bit string represents a valid element of the set U .
 - (b) *Unique representation*, i.e., given any arbitrary set element $u \in U$, compute a string $\hat{u}$ that canonically represents u .
3. There exists a distinguished element $u_0 \in U$, called the *origin*, such that its bit-string representation is known.
4. There exists an efficient algorithm that, given (some bit-string representations of) any $g \in G$ and any $u \in U$, outputs $g \star u$.

2.3 Cryptographic Hardness Assumptions

The security proofs of our constructions depend on the following two cryptographic hard problems. The first one is the well-known Decisional Diffie-Hellman (DDH) problem. Security of the classical instantiation of our construction is based on it. The second is a generalization of the DDH problem in the group action setting. Security of the post-quantum instantiation of our construction is based on it.

Definition 1 (DDH Assumption). *Let $\mathbb{G}$ be a finite cyclic group of prime order p and let g be a generator of the cyclic group which is uniformly sampled from $\mathbb{G}$. We say that the DDH problem is hard for the group $\mathbb{G}$, if for any probabilistic polynomial time algorithm $\mathcal{A}$,*

$$\text{Adv}_{\mathcal{A}}^{\text{DDH}}(\lambda) \triangleq |\Pr[\mathcal{A}(g, g^\alpha, g^\beta, g^{\alpha\beta}) = 1] - \Pr[\mathcal{A}(g, g^\alpha, g^\beta, g^\gamma) = 1]| \leq \text{negl}(\lambda),$$

where $\alpha, \beta, \gamma \xleftarrow{\$} \mathbb{Z}_p^*$ and the group $\mathbb{G}$ (and consequently, its order p and its generator) depend on the security parameter λ .

Definition 2 (Group Action DDH (GA-DDH) Assumption). *Let $(G, U, \star)$ be an EGA as defined above. We say that the DDH assumption holds with respect to a group action $(G, U, \star)$ if for any security parameter λ such that $|G| = \omega(\lambda^c)$ for any $c = O(1)$, for any $g, h, k \leftarrow G$, for any $u \leftarrow U$, and any PPT adversary Adv , we have*

$$\text{Adv}_{\mathcal{A}}^{\text{GA-DDH}}(\lambda) \triangleq |\Pr[\mathcal{A}(u, g \star u, h \star u, (g \cdot h) \star u) = 1] - \Pr[\mathcal{A}(u, g \star u, h \star u, k \star u) = 1]| \leq \text{negl}(\lambda).$$

2.4 Dynamic Searchable Symmetric Encryption Schemes

A dynamic searchable symmetric encryption (DSSE) scheme consists of three interactive algorithms: Setup, Update, and Search. We adopted the following definition of DSSE from Chamani et al. [13].

- The **Setup** algorithm takes as input the security parameter λ and the database DB and outputs a state σ , which is kept secret, and an encrypted database EDB, which is made public.
- The **Update** algorithm is interactive. First, the client takes as input the current state σ , the operation $op \in \{add, del\}$, and a keyword-identifier pair (w, id) and outputs an update token UT, which is sent to the server. On input UT, the server updates the encrypted database EDB and the client state σ also gets updated.
- The **Search** algorithm is also an interactive one. Here, the client first takes as input the current state σ and a query q and outputs a search token ST, which is sent to the server. The server uses the search token ST and the encrypted database EDB to retrieve the desired encrypted identifiers and send the identifiers to the client. The client finally outputs the list of file identifiers.

2.5 Security of DSSE

To understand the security of a DSSE scheme, we first define the notion of a leakage function. Formally, the leakage function for any DSSE scheme is given by

$$\mathcal{L} \triangleq (\mathcal{L}^{\text{Setup}}, \mathcal{L}^{\text{Update}}, \mathcal{L}^{\text{Search}}),$$

where $\mathcal{L}^{\text{Setup}}$, $\mathcal{L}^{\text{Update}}$ and $\mathcal{L}^{\text{Search}}$ denotes the leakage to a semi-honest adversarial server $\mathcal{A}$ during the Setup, Update and Search algorithms respectively. With this, the security model of a DSSE scheme is given by two experiments, namely **Real** and **Ideal**. The experiments are defined as follows:

Real World : On input the security parameter λ and a database DB, the real challenger first returns an encrypted database EDB following the Setup algorithm of the scheme. Then $\mathcal{A}$ interacts with the real challenger with update or search queries. The challenger responds to each of these queries following the Update and Search algorithms of the scheme. Finally, $\mathcal{A}$ outputs a single bit b .

Ideal World : Here the encrypted database EDB is simulated by a stateful probabilistic polynomial time simulator $\mathcal{S}$ in spite of the real challenger following the setup leakage $\mathcal{L}^{\text{Setup}}$ of the scheme. To respond to the update or search queries of the adversary $\mathcal{A}$, the simulator $\mathcal{S}$ uses the update leakage $\mathcal{L}^{\text{Update}}$ or the search leakage $\mathcal{L}^{\text{Search}}$, respectively. Finally, $\mathcal{A}$ outputs a single bit b .

Definition 3 (DSSE Security). A DSSE scheme is said to be adaptively secure with respect to a leakage function $\mathcal{L}$ if for any security parameter $\lambda \in \mathbb{N}$ and for any probabilistic polynomial time adversary $\mathcal{A}$, there exists a probabilistic polynomial-time simulator $\mathcal{S}$ such that

$$|\Pr[\text{Real}_{\mathcal{A}}(\lambda) = 1] - \Pr[\text{Ideal}_{\mathcal{A}, \mathcal{S}}(\lambda) = 1]| \leq \text{negl}(\lambda).$$

3 Forward and Backward Privacy of Conjunctive DSSE

In this section, we revisit the security definition of forward and backward privacy for single-keyword DSSE from [5,6,15]. We then introduce a new notion of backward privacy for conjunctive DSSE.

3.1 Forward and Backward Privacy of Single-keyword DSSE

Let $\mathcal{Q}$ list all queries the adversarial server issues. Note that queries can be of the following two types: (i) an update query at timestamp t , involving keyword w on file identifier id , denoted as $(t, op, (w, id))$, where $op \in \{add, del\}$, (ii) a search query for a keyword w at timestamp t , denoted as (t, w) . The following are different leakage functions associated with search and update queries.

sp(w): The search pattern leaks the repetition of search queries on keyword w .

$$\text{sp}(w) \triangleq \{t : (t, w) \in \mathcal{Q}\}$$

TimeDB(w): TimeDB(w) leaks the (t, id) pairs such that the keyword w has been added to the file identifier id at timestamp t , and has further not been removed.

$$\text{TimeDB}(w) \triangleq \{(t, id) : (t, \text{add}, (w, id)) \in \mathcal{Q} \wedge \forall t' > t, (t', \text{del}, (w, id)) \notin \mathcal{Q}\}.$$

Updates(w): At any instance, Updates(w) contains the timestamps at which all the updates on w has occurred.

$$\text{Updates}(w) \triangleq \{t : (t, \text{op}, (w, id)) \in \mathcal{Q}\}.$$

DelHist(w): At any instance, DelHist(w) contains the pair (t^+, t^-) such that w has been added in some file at timestamp t^+ and later has been deleted from it at timestamp t^- .

$$\text{DelHist}(w) \triangleq \{(t^+, t^-) : \exists id \text{ such that } (t^+, \text{add}, (w, id)) \in \mathcal{Q} \wedge (t^-, \text{del}, (w, id)) \in \mathcal{Q}\}.$$

With these definitions of leakage functions, we now discuss forward privacy and three types of backward privacy for single-keyword DSSE. For the rest of the section, $\mathcal{L}'$ and $\mathcal{L}''$ are stateless functions, and update queries are restricted to adding or deleting a single keyword-identifier pair.

Forward Privacy: An adaptively secure DSSE scheme Σ with corresponding leakage function $\mathcal{L}_\Sigma$ is forward private iff $\mathcal{L}_\Sigma^{\text{Update}}$ can be written as:

$$\mathcal{L}_\Sigma^{\text{Update}}(t, \text{op}, (w, id)) = \mathcal{L}'(\text{op}, id),$$

Type-I Backward Privacy: An adaptively secure DSSE scheme Σ with corresponding leakage function $\mathcal{L}_\Sigma$ is insertion pattern revealing backward private (or Type-I backward private) iff $\mathcal{L}_\Sigma^{\text{Update}}$ and $\mathcal{L}_\Sigma^{\text{Search}}$ can be written as:

$$\begin{aligned} \mathcal{L}_\Sigma^{\text{Update}}(t, \text{op}, (w, id)) &= \mathcal{L}'(\text{op}), \\ \mathcal{L}_\Sigma^{\text{Search}}(t, w) &= \mathcal{L}''(\text{sp}(w), \text{TimeDB}(w), c_w), \end{aligned}$$

where c_w is the number of updates on keyword w .

Type-II Backward Privacy: An adaptively secure DSSE scheme with corresponding leakage function $\mathcal{L}$ is update pattern revealing backward private (or Type-II backward private) iff $\mathcal{L}_\Sigma^{\text{Update}}$ and $\mathcal{L}_\Sigma^{\text{Search}}$ can be written as:

$$\begin{aligned} \mathcal{L}_\Sigma^{\text{Update}}(t, \text{op}, (w, id)) &= \mathcal{L}'(\text{op}, w), \\ \mathcal{L}_\Sigma^{\text{Search}}(t, w) &= \mathcal{L}''(\text{sp}(w), \text{TimeDB}(w), \text{Updates}(w)). \end{aligned}$$

Type-III Backward Privacy: A adaptively secure DSSE scheme with corresponding leakage function $\mathcal{L}$ is weakly backward private (or Type-III backward private) iff $\mathcal{L}_\Sigma^{\text{Update}}$ and $\mathcal{L}_\Sigma^{\text{Search}}$ can be written as:

$$\begin{aligned} \mathcal{L}_\Sigma^{\text{Update}}(t, \text{op}, (w, id)) &= \mathcal{L}'(\text{op}, w), \\ \mathcal{L}_\Sigma^{\text{Search}}(t, w) &= \mathcal{L}''(\text{sp}(w), \text{TimeDB}(w), \text{Updates}(w), \text{DelHist}(w)). \end{aligned}$$

It is obvious that in terms of privacy, a construction achieving Type-I backward privacy is the strongest one, while a construction achieving Type-III is the weakest one. Now assume for two notions of privacy A and B, $A \prec B$ represents A is a weaker notion than B. In other words, a scheme achieving security A leaks more information as compared to a scheme achieving security B. Thus, we can write the following relation with respect to backward privacy:

$$\text{Type-III} \prec \text{Type-II} \prec \text{Type-I}.$$

3.2 Understanding the Leakage from Conjunctive Queries

Having defined the backward privacy of the single keyword DSSE schemes, our target now is to extend the definitions to the general backward privacy notion for conjunctive DSSE schemes. For that, let us consider that at any instance $\mathcal{Q}$ is the list of all queries issued by an adversarial server executing any conjunctive DSSE scheme with a client. The update query is the same as the single-keyword case above. In place of a single keyword search, a search query for the conjunction of keywords $w_1, w_2, \dots, w_n$ at timestamp t is denoted by (t, q) where $q = w_1 \wedge w_2 \wedge \dots \wedge w_n$. By convention, we always consider w_1 as the keyword with the least number of updates. Also for a conjunctive query q , we denote $W^q = \{w_1, w_2, \dots, w_n\}$ as the set of keywords involved in the query. Before defining the formal notion of backward privacy for conjunction, we introduce the following leakage functions. Each of these functions is the most intuitive extension of the leakage functions from single-keyword settings and captures a wide range of leakage components related to conjunctive DSSE schemes, except the SizeC leakage. The SizeC leakage is independent of the other leakage functions and comes into consideration only for conjunctive DSSE schemes.

Search Patterns: We consider the following two notations for search patterns.

- (i) $\text{sp}(q)$: This search pattern leaks the repetition on each keyword $w \in q$. Formally, we have:

$$\text{sp}(q) = \bigcup_{w \in W^q} \text{sp}(w).$$

- (ii) $\text{sp}^-(q)$: This restricted search pattern only leaks the search pattern of the previous searches that have the same s-term as q . Formally,

$$\text{sp}^-(q) = \{t : \exists (t, q') \in \mathcal{Q} \wedge \text{sterm}(q') = w_1\}$$

Note that $\text{sp}^-(q)$ leakage is strictly less than $\text{sp}(w_1)$ since it only leaks the timestamps of those search queries whose s-term is w_1 . Thus, if there is a search query at some timestamp t' with w_1 as x-term, then $t' \in \text{sp}(w_1)$ but $t' \notin \text{sp}^-(q)$.

Output Leakage: During any conjunctive query q , the output leakage function TimeDB(q) is defined to be the list of all file identifiers in the conjunction along

with the corresponding $\{t_i\}_{i=1}^n$ containing insertion timestamps for each keyword in the conjunction. Here, t_i indicates the timestamp when keyword w_i was inserted into the file with identifier id_i .

$$\text{TimeDB}(q) \triangleq \left\{ \left(\{t_i\}_{i=1}^n, id \right) : \forall i \left[(t_i, \text{add}, (w_i, id)) \in \mathcal{Q} \wedge \forall t' > t_i, (t', \text{del}, (w_i, id)) \notin \mathcal{Q} \right] \right\}.$$

Note that an adversary only learns the set of all timestamps, but not the exact mapping of keywords to those timestamps. This definition is adapted from [33].

Update Leakage: We consider two types of update leakage components as described below:

- (i) **UpdatesC**(q): During any conjunctive search query q , **UpdatesC**(q) leaks all the timestamps of all the updates involving each searched keyword w_i .

$$\text{UpdatesC}(q) \triangleq \bigcup_{i=1}^n \text{Updates}(w_i).$$

This definition is equivalent to the definition of **Time**(q) leakage from [49] and extends the **Updates**(w) leakage of the single-keyword case. We believe that **UpdatesC** is a more intuitive name for this leakage function as it motivates the trivial extension from the single keyword case.

- (ii) **UpdatesC**⁻(q): The leakage function **UpdatesC**⁻(q) reveals (i) the timestamps of all updates associated with the search term w_1 , and (ii) all pairs of timestamps corresponding to updates of the *stern* w_1 and the associated *xterms* that occur in the same file identifier id and involve the same update operation op . To formally define this leakage, let us define the following leakage

$$\text{Updates}(w_1, w_i) = \{(t_1, t_i) \mid \exists (op, id) : (t_1, op, (id, w_1)) \wedge (t_i, op, (id, w_i)) \in \mathcal{Q}\}.$$

Now we define the **UpdatesC**⁻(q) leakage as follows:

$$\text{UpdatesC}^-(q) \triangleq \text{Updates}(w_1) \cup \left(\bigcup_{i=1}^n \text{Updates}(w_1, w_i) \right).$$

It is easy to see that, by definition, **UpdatesC**⁻(q) leaks less than what **UpdatesC**(q) leaks.

Deletion Leakage: During any conjunctive search query q , the deletion leakage is captured by the leakage function **DelHistC**(q) that leaks the pair of timestamps (t_i^+, t_i^-) such that there exists some $w_i \in W^q$ such that w_i is inserted to some file at timestamp t_i^+ and later deleted from it at timestamp t_i^- . Formally,

$$\text{DelHistC}(q) \triangleq \left\{ (t_i^+, t_i^-) : \exists w_i, id \text{ such that } (t_i^+, \text{add}, (w_i, id)) \in \mathcal{Q} \wedge (t_i^-, \text{del}, (w_i, id)) \in \mathcal{Q} \right\}.$$

This is a completely new leakage function, a trivial extension to **DelHist**(w) leakage of the single keyword case.

Size Leakage: During a conjunctive search query q , a protocol can leak information about the number of matching files for some of the conjunctive sub-queries. Depending on the amount of information leak, we define the following three types of **SizeC** leakage.

$$\text{SizeC-I}(q) \triangleq \{ |\text{DB}(w_1) \cap \text{DB}(w_j)| : j \in [2, n] \}$$

where w_1 is the current least frequent search keyword.

Now consider for some $q^* \in \mathcal{Q}$, w^* denotes the corresponding least frequent search keyword. Then we define,

$$\text{SizeC-II}(q) \triangleq \{ |\text{DB}(w^*) \cap \text{DB}(w_j)| : \exists q^* \in \mathcal{Q} : w_j \in W^q \cap W^{q^*} \}.$$

Finally, we define the weakest notion for **SizeC** leakage, which leaks the number of matching files for any possible sub-conjunction.

$$\text{SizeC-III}(q) \triangleq \{ |\text{DB}(w_i) \cap \text{DB}(w_j)| : i, j \in [2, n] \ \& \ i \neq j \}.$$

These new definitions formalize and generalize the definition of size leakage components provided in [47], which were specific to the SDSSE-CQ and SDSSE-CQ-S constructions.

3.3 New Backward Privacy Notion for Conjunctive DSSE

We now introduce three security notions of backward privacy for conjunctive search queries. It is worth mentioning that the definition is a very intuitive extension of the same notion for single-keyword DSSE and efficiently captures all the leakage components corresponding to any conjunctive DSSE scheme defined to date. As previous $\mathcal{L}'$ and $\mathcal{L}''$ are stateless functions.

C1 (Conjunctive Type-I) Backward Privacy : An adaptively secure DSSE scheme Σ with corresponding leakage function $\mathcal{L}_\Sigma$ is combined insertion time revealing backward private (or C1 backward private) iff $\mathcal{L}_\Sigma^{\text{Update}}$ and $\mathcal{L}_\Sigma^{\text{Search}}$ can be written as:

$$\begin{aligned} \mathcal{L}_\Sigma^{\text{Update}}(t, op, (w, id)) &= \mathcal{L}'(op), \\ \mathcal{L}_\Sigma^{\text{Search}}(t, q) &= \mathcal{L}''\left(\text{sp}(q), \text{TimeDB}(q), \sum_{i=1}^n c_{w_i}\right). \end{aligned}$$

Here c_{w_i} denotes the number of updates corresponding to keyword w_i . In a nutshell, during any search operation for a query q , the notion C1 leaks the list of all file identifiers matching the conjunction, corresponding insertion times, and the total number of updates across all searched keywords, but not all c_{w_i} individually. Note that, $\text{TimeDB}(q)$ boils down to $\text{TimeDB}(w)$ when $q = w$. Hence, our definition of C1 backward privacy naturally extends the notion of Type-I backward privacy.

C2 (Conjunctive Type-II) Backward Privacy. An adaptively secure DSSE scheme Σ with corresponding leakage function $\mathcal{L}_\Sigma$ is combined update time revealing backward private (or C2 backward private) iff $\mathcal{L}_\Sigma^{\text{Update}}$ and $\mathcal{L}_\Sigma^{\text{Search}}$ can be written as:

$$\begin{aligned}\mathcal{L}_\Sigma^{\text{Update}}(t, op, (w, id)) &= \mathcal{L}'(op), \\ \mathcal{L}_\Sigma^{\text{Search}}(t, q) &= \mathcal{L}''(\text{sp}(q), \text{TimeDB}(q), \text{UpdatesC}(q)).\end{aligned}$$

Informally, during any search query q , along with the leakage components of C1, C2 additionally leaks the timestamps of all the update operations corresponding to each w_i .

Note that, for $q = w$, $\text{UpdatesC}(q) = \text{Updates}(w)$. Hence, our definition of C2 backward privacy naturally extends the notion of Type-II backward privacy for single-keyword DSSE.

C3 (Conjunctive Type-III) Backward Privacy : An adaptively secure DSSE scheme with corresponding leakage function $\mathcal{L}_\Sigma$ is C3 combined weak backward private iff $\mathcal{L}_\Sigma^{\text{Update}}$ and $\mathcal{L}_\Sigma^{\text{Search}}$ can be written as:

$$\begin{aligned}\mathcal{L}_\Sigma^{\text{Update}}(t, op, (w, id)) &= \mathcal{L}'(op), \\ \mathcal{L}_\Sigma^{\text{Search}}(t, q) &= \mathcal{L}''(\text{sp}(q), \text{TimeDB}(q), \text{UpdatesC}(q), \text{DelHistC}(q)).\end{aligned}$$

Along with the leakage components corresponding to C2, C3 also leaks the information *which deletion cancels out which addition* for any searched keyword w_i . Since for $q = w$, $\text{DelHistC}(q) = \text{DelHist}(w)$. Hence, our definition of C3 backward privacy naturally extends the notion of Type-III backward privacy.

The above definitions cover all common leakage components for any conjunctive DSSE scheme. However, certain conjunctive DSSE schemes additionally leak information about the number of matching files for some sub-conjunctions. To capture this additional leakage, we define three additional types of SizeC leakage functions in Section 3.2, which are independent of the other leakage components. To measure the level of backward privacy for any conjunctive DSSE scheme, we first identify where its leakage profile sits in our new backward privacy notion, and whenever there is any SizeC leakage, we explicitly add the corresponding level of SizeC leakage to the common backward privacy guarantee. For example, if some conjunctive DSSE scheme Σ guarantees C2 backward privacy and its size leakage is subsumed by SizeC-II, then we say Σ is C1-II backward private.

Next, we introduce another notion of backward privacy, which is particularly relevant for OXT based constructions. This notion is a similar but slightly stronger variant of C2 backward privacy as described below:

C2⁻ (Conjunctive Strong Type-II) Backward Privacy : This notion is close to C2 backward privacy; however, it has a reduced search pattern and update leakage components. To be precise, for a conjunctive query q , the search pattern only leaks the search pattern of the previous searches that have the same s-term as q , and the update function leaks (i) all the update operations corresponding to

the s-term, and (ii) a restricted number of update operations corresponding to the x-terms. Formally, an adaptively secure DSSE scheme Σ with corresponding leakage function $\mathcal{L}_\Sigma$ is combined update time revealing backward private (or strong C2 backward private) iff $\mathcal{L}_\Sigma^{\text{Update}}$ and $\mathcal{L}_\Sigma^{\text{Search}}$ can be written as:

$$\begin{aligned}\mathcal{L}_\Sigma^{\text{Update}}(t, op, (w, id)) &= \mathcal{L}'(op), \\ \mathcal{L}_\Sigma^{\text{Search}}(t, q) &= \mathcal{L}''(\text{sp}^-(q), \text{TimeDB}(q), \text{UpdatesC}^-(q)).\end{aligned}$$

Note that, for $q = w$, $\text{sp}^-(q) = \text{sp}(w)$, $\text{UpdatesC}^-(q) = \text{Updates}(w)$, and hence, this notion can also be considered as an extension of the notion of Type-II backward privacy for single-keyword DSSE.

3.4 Analyzing Prior Schemes

It is evident that, for conjunctive DSSE, C1 and C3 represent the strongest and weakest notions of backward privacy, respectively. In other words, we have a privacy *ranking* of the form

$$\text{C3} \prec \text{C2} \prec \text{C2}^- \prec \text{C1}.$$

Analyzing prior conjunctive DSSE schemes with respect to the above notions of backward privacy yields the following observations:

- ODXT [33], achieves C2^- backward privacy.
- Both FBDSSE-CQ and SFBDSSE-CQ from [49] achieve C2 backward privacy.
- SDSSE-CQ and SDSSE-CQ-S from [47] achieve C2-II and C2-I backward privacy, respectively.

Overall, ODXT satisfies the strongest notion of backward privacy. This motivates us to build a *conjunctive* DSSE scheme that simultaneously achieves forward privacy (unlike ODXT) and a notion of backward privacy that is similar to ODXT (i.e., stronger than the notions of backward privacy achieved by FBDSSE-CQ, SFBDSSE-CQ, SDSSE-CQ, and SDSSE-CQ-S).

4 A Framework for Forward and Strongly Backward Private Conjunctive DSSE

In this section, we present fp-GA-ODXT – a new framework for designing conjunctive DSSE schemes that simultaneously achieve forward privacy and a notion of backward privacy that is similar to ODXT. Our framework is based on effective group actions and injective trapdoor functions.

Algorithm 1 : GA-ODXT.Update ($op, (w, id)$)

```
1: function CLIENT
2:    $addr \leftarrow F(K_T, w \| c_w \| 0)$ 
3:    $val \leftarrow F(K_T, w \| c_w \| 1) \oplus id \| op$ 
4:    $\alpha \leftarrow F_G(K_Y, id \| op) \cdot (F_G(K_Z, w \| c_w))^{-1}$ 
5:    $xtag \leftarrow (F_G(K_X, w) \cdot F_G(K_Y, id \| op)) \star u$ 
6:   Send ( $addr, val, \alpha, xtag$ ) to Server
```

```
1: function SERVER
2:   TSet[ $addr$ ] = ( $val, \alpha$ )
3:   XSet[ $xtag$ ] = 1
```

4.1 GA-ODXT: An Adaptation of ODXT to Group Actions

We first adapt the construction ODXT to the group action setting. Note that ODXT is based on $\mathbb{G}$, a cyclic group of prime order p , a generator g where the DDH assumption holds. Here we consider the group action adaptation of ODXT, dubbed GA-ODXT, that works on an EGA $(G, U, \star)$ as defined in Section 2.2. We consider $|G| = |U| = O(2^\lambda)$, where λ is the security parameter. Let u be an element uniformly sampled from the set U . We assume u to be public. The construction is built on two PRFs $F : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ and $F_G : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow G$. One may wonder if there exists such a PRF F_G with the above-mentioned domain and range. In practice, one can always construct F_G by simply taking the output of the PRF F and hashing it into the group G . This is possible because of its dense representation. To keep things simple, we leave out these technical details. The construction is based on the GA-DDH assumption (Definition 2). The update and search operations of the GA-ODXT construction are described in Algorithm 1 and 2, respectively.

4.2 Issues with Forward Privacy in GA-ODXT

Similar to ODXT, GA-ODXT is not forward private. Intuitively, failing to provide forward privacy means that the series of computations does not guarantee that the search operation cannot link to any future update operation on the x-terms of that search. Let us consider the following example that depicts that GA-ODXT does not preserve forward privacy. Consider a database where the keyword w_1 is present in the file id_1 , and the keyword w_2 is present in the files id_2 and id_3 , but not in id_1 . Suppose the client performs a search operation $w_1 \wedge w_2$, where w_1 is the least frequent keyword, i.e., the s-term. Of course, the search result will return an empty set. However, during this search operation, the server gets to learn

$$xtag = (F_G(K_X, w) \cdot F_G(K_Y, id \| op)) \star u ,$$

which is currently not a valid one. Later, at some point, if w_2 is added to the file id_1 , then during the update operation, the client will generate the same $xtag$ and send it to the server. Upon receiving this $xtag$ from the client, the server

Algorithm 2 : GA-ODXT.Search ($w_1 \wedge w_2 \wedge \dots \wedge w_n$)

```

1: function CLIENT (ROUND 1)
2:   for  $c = 1, 2, \dots, c_{w_1}$  do
3:      $\text{saddr}_c \leftarrow F(K_T, w \| c \| 0)$ 
4:     for  $i = 2, 3, \dots, n$  do
5:        $\text{xtoken}_{i,c} \leftarrow (F_G(K_X, w_i) \cdot F_G(K_Z, w \| c)) \star u$ 
6:   Send ( $\{\text{saddr}_c\}_{c=1}^{c_{w_1}}, \{\{\text{xtoken}_{i,c}\}_{i=2}^n\}_{c=1}^{c_{w_1}}$ ) to Server
7: function SERVER
8:   for  $c = 1, 2, \dots, c_{w_1}$  do
9:      $\text{cnt}_c \leftarrow 1$ 
10:     $(\text{val}_c, \alpha_c) \leftarrow \text{TSet}[\text{saddr}_c]$ 
11:    for  $i = 2, 3, \dots, n$  do
12:       $\text{xtag}_{i,c} = \alpha_c \star \text{xtoken}_{i,c}$ 
13:      if  $\text{XSet}[\text{xtag}_{i,c}] = 1$  then
14:         $\text{cnt}_c \leftarrow \text{cnt}_c + 1$ 
15:    Send ( $\{c, \text{val}_c, \text{cnt}_c\}_{c=1}^{c_{w_1}}$ ) to Client
16: function CLIENT (ROUND 2)
17:   for  $c = 1, 2, \dots, c_{w_1}$  do
18:      $(\text{id}_c, \text{op}) \leftarrow F(K_T, w_1 \| c \| 1) \oplus \text{val}_c$ 
19:     if  $\text{op} = \text{add} \wedge \text{cnt}_c = n$  then  $\text{Res} \leftarrow \text{Res} \cup \{\text{id}_c\}$ 
20:     else  $\text{Res} \leftarrow \text{Res} \setminus \{\text{id}_c\}$ 
21:   Output Res

```

can check that this has already been computed in a previous search operation. Hence, it can deduce the information that “the keyword that is updated in a file was a part of the previous conjunctive search operation”, which is not expected in a forward private DSSE scheme. The core problem for this attack lies in the fact that the xtag values do not depend on the update operation’s timestamp of occurrence. In other words, what we expect is that the server should not be able to recompute a valid xtag until the client has previously computed that during some update.

A Naïve Solution. It is clear from the above discussion that we need to consider the update counter or update timestamp while generating xtag values during updates. A naive approach to do so would be to add the current update count c_w of the keyword w in the xtag values as follows:

$$\text{xtag} \leftarrow (F_G(K_X, w \| c_w) \cdot F_G(K_Y, \text{id} \| \text{op})) \star u.$$

Consequently, we would require the generation of xtoken to be modified as:

$$\text{xtoken}_{i,c} \leftarrow (F_G(K_X, w_i \| c_i) \cdot F_G(K_Z, w_1 \| c)) \star u, \forall c_i \in [c_{w_i}], c \in [c_{w_1}].$$

This should resolve the forward-privacy issue, as the xtag values for a keyword now depend on the keyword’s update count. So, for the server reconstruction of xtag value, it now largely depends on whether the update has occurred before

the search or not. However, this doesn't come for free - as evident from the above modification, the client now needs to send c_{w_i} many xtoken values for each x -term w_i and each update on s -term w_1 . This essentially amplifies the client-side search communication cost to $O(c_{w_1} \cdot \sum_{i=2}^n c_{w_i})$, which is identical to the cost of performing a single keyword query for each keyword in the conjunction.

4.3 The fp-GA-ODXT Framework

The above discussion motivates us to extend GA-ODXT to achieve forward privacy without increasing the overhead of the client computation and communication. Specifically, our primary objective is to include the current update count for each keyword during updates to that keyword, while doing so in a feasible, practical way. That is, we want the client-side computational cost not to blow up during conjunctive search queries as a result of the modification to the update. We generically address this problem and consider a function f_{ow} that satisfies the following properties.

- (P1) For each keyword w , client should be able to efficiently compute $f_{\text{ow}}(w, \cdot)$
- (P2) For each keyword w , the server should be able to efficiently compute $f_{\text{ow}}(w, c)$ from the knowledge of $f_{\text{ow}}(w, c + 1)$. That is, backward computation should be easy for the server.
- (P3) For each keyword w , the server should not be able to efficiently compute $f_{\text{ow}}(w, c + 1)$ from $f_{\text{ow}}(w, c)$, i.e., forward computation should be hard for the server.

Now, given such a function, we define the cross tag during an update operation as follows:

$$\text{xtag} \leftarrow (F_G(K_X, w) \cdot F_G(K_Y, id || op) \cdot f_{\text{ow}}(w, j)) \star u,$$

where j is the number of update on the keyword w . Now, during the conjunctive search for $(w_1 \wedge w_2 \wedge \dots \wedge w_n)$, the xtoken values corresponding to x -term w_i are computed as:

$$\text{xtoken}_{i,c} \leftarrow F_G(K_X, w_i) \cdot F_G(K_Z, w_1 || c) \star u,$$

where $i \in \{2, 3, \dots, n\}$ and $c \in [c_{w_1}]$. The server computes all the possible xtag values as:

$$(\alpha_c \cdot f_{\text{ow}}(w_i, c)) \star \text{xtoken}_{i,c},$$

for each $i \in \{2, 3, \dots, n\}$ and $c \in [c_{w_i}]$ and checks whether any of these reconstructed values are valid or not. This is easy for the server with the knowledge of $\text{xtoken}_{i,c}$ and previously stored α_c values. Note that property (P1) and (P2) of f_{ow} ensures that the server can compute all of $f_{\text{ow}}(w_i, c_{w_i}), f_{\text{ow}}(w_i, c_{w_i} - 1), \dots, f_{\text{ow}}(w_i, 1)$ from the knowledge of $f_{\text{ow}}(w_i, c_{w_i})$. However, since the server can not compute $f_{\text{ow}}(w_i, c_{w_i} + \ell)$ from $f_{\text{ow}}(w_i, c_{w_i})$ for any $i \in \{2, 3, \dots, n\}$ and $\ell \geq 1$, by property (P3), it fails to link this search operation to any future update

Algorithm 3 : fp-GA-ODXT.Update ($op, (w, id)$)

```
1: function CLIENT
2:    $\text{addr} \leftarrow F(K_T, w \| c_w \| 0)$ 
3:    $\text{val} \leftarrow F(K_T, w \| c_w \| 1) \oplus id \| op$ 
4:    $\alpha \leftarrow F_G(K_Y, id \| op) \cdot F_G(K_Z, w \| c_w)^{-1}$ 
5:    $\text{xtag} \leftarrow (F_G(K_X, w) \cdot F_G(K_Y, id \| op) \cdot \mathbf{f}_{\text{ow}}(w, c_w)) \star u$ 
6:   Send ( $\text{addr}, \text{val}, \alpha, \text{xtag}$ ) to Server

1: function SERVER
2:   TSet[ $\text{addr}$ ]  $\leftarrow (\text{val}, \alpha)$ 
3:   XSet[ $\text{xtag}$ ]  $\leftarrow 1$ 
```

operation on w_i . Now, let us see the correctness of the construction. Suppose the keyword w_1 and w_i were added to the identifier id at their c^{th} and j^{th} update operation, respectively. Then, during the search operation, the server computes the j^{th} xtag corresponding to w_i as:

$$\begin{aligned} \text{xtag}'_{i,j} &= (\alpha_c \cdot \mathbf{f}_{\text{ow}}(w_i, j)) \star \text{xtoken}_{i,c} \\ &= ((F_G(K_Y, id \| op) \cdot F_G(K_Z, w_1 \| c)^{-1}) \cdot \mathbf{f}_{\text{ow}}(w_i, j)) \star \\ &\quad (F_G(K_X, w_i) \cdot F_G(K_Z, w_1 \| c) \star u) \\ &= (F_G(K_X, w_i) \cdot F_G(K_Y, id \| op) \cdot \mathbf{f}_{\text{ow}}(w_i, j)) \star u. \end{aligned}$$

where the last equality follows from the compatibility of the group action.

Improving the Communication Complexity. Next, we observe that during any search operation of the form $(w_1 \wedge \dots \wedge w_n)$, the client computes and sends c_{w_1} many x-tokens corresponding to each x-term (i.e., w_2 to w_n), essentially making the communication complexity during a search operation $n \cdot c_{w_1}$. So, we investigate whether it is possible to reduce the communication complexity by sending a single x-token for each x-term, along with auxiliary information, so that the remaining x-token generation is delegated to the server. We answer this affirmatively by using the concept of s-tokens that essentially helps the server to generate x-token $\text{xtoken}_{i,j}$ from $\text{xtoken}_{i,j-1}$ and stoken_j , for every x-term i and any $j \in [c_{w_1}]$. We present the update and search operations of GA-ODXT in Algorithms 3 and 4, respectively. Note that, in Algorithm 4, the public computation of $\mathbf{f}_{\text{ow}}(w_i, j-1)$ from $\mathbf{f}_{\text{ow}}(w_i, j)$ is denoted as $\mathbf{f}_{\text{ow}}(w_i, j-1) = \phi(\mathbf{f}_{\text{ow}}(w_i, j))$.

It is easy to see the client's computation and communication overhead for an update and a search operation for some conjunction of n keywords is $O(1)$ and $O(n + c_{w_1})$, respectively.

4.4 Security of fp-GA-ODXT

The leakage profile of fp-GA-ODXT is given by

$$\mathcal{L}_{\text{fp-GA-ODXT}} = \left(\mathcal{L}_{\text{fp-GA-ODXT}}^{\text{Setup}}, \mathcal{L}_{\text{fp-GA-ODXT}}^{\text{Update}}, \mathcal{L}_{\text{fp-GA-ODXT}}^{\text{Search}} \right),$$

Algorithm 4 : fp-GA-ODXT.Search ($w_1 \wedge w_2 \wedge \dots \wedge w_n$)

```

1: function CLIENT (ROUND 1)
2:   for  $c = c_{w_1}, (c_{w_1} - 1), \dots, 1$  do
3:      $\text{saddr}_c \leftarrow F(K_T, w_1 \| c \| 0)$ 
4:      $\text{stoken}_c \leftarrow F_G(K_Z, w_1 \| c)^{-1} \cdot F_G(K_Z, w_1 \| c - 1)$ 
5:   for  $i = 2, 3, \dots, n$  do
6:      $\text{xtoken}_{i, c_{w_i}} \leftarrow (F_G(K_X, w_i) \cdot F_G(K_Z, w_1 \| c_{w_1})) \star u$ 
7:   Send ( $\{\text{saddr}_c\}_{c=1}^{c_{w_1}}, \{\text{xtoken}_{i, c_{w_i}}\}_{i=2}^n, \{\text{stoken}_i\}_{i=1}^{c_{w_1}}, \{\text{fow}(w_i, c_{w_i})\}_{i=2}^n$ ) to Server

1: function SERVER
2:   for  $c = c_{w_1}, (c_{w_1} - 1), \dots, 1$  do
3:      $\text{cnt}_c \leftarrow 1$ 
4:      $(\text{val}_c, \alpha_c) \leftarrow \text{TSet}[\text{saddr}_c]$ 
5:     for  $i = 2, 3, \dots, n$  do
6:       for  $j = c_{w_i}, (c_{w_i} - 1), \dots, 1$  do
7:          $\text{xtag}' = (\alpha_c \cdot \text{fow}(w_i, j)) \star \text{xtoken}_{i, c}$ 
8:         if  $\text{XSet}[\text{xtag}'] = 1$  then
9:            $\text{cnt}_c \leftarrow \text{cnt}_c + 1$ 
10:        break
11:        $\text{fow}(w_i, j - 1) \leftarrow \phi(\text{fow}(w_i, j))$ 
12:        $\text{xtoken}_{i, c-1} \leftarrow (\text{stoken}_c) \star \text{xtoken}_{i, c}$ 
13:   Send ( $\{c, \text{val}_c, \text{cnt}_c\}_{c=1}^{c_{w_1}}$ ) to Client

1: function CLIENT (ROUND 2)
2:   for  $c = 1, 2, \dots, c_{w_1}$  do
3:      $\text{id}_c \| \text{op} \leftarrow F(K_T, w \| c \| 1) \oplus \text{val}_c$ 
4:     if  $\text{op} = \text{add} \wedge \text{cnt}_c = n$  then  $\text{Res} \leftarrow \text{Res} \cup \{\text{id}_c\}$ 
5:     else  $\text{Res} \leftarrow \text{Res} \setminus \{\text{id}_c\}$ 
6:   Output Res

```

where each leakage is described below.

$$\begin{aligned}
\mathcal{L}_{\text{fp-GA-ODXT}}^{\text{Setup}}(\mathcal{Q}) &= \mathcal{L}_{\text{fp-GA-ODXT}}^{\text{Update}}(\mathcal{Q}) = \emptyset, \\
\mathcal{L}_{\text{fp-GA-ODXT}}^{\text{Search}}(\mathcal{Q}) &= (\text{sp}^-(q), \text{TimeDB}(q), \text{Updates}^-(q)).
\end{aligned}$$

Observe that the leakage profile defined above satisfies the definition of C2^- -backward privacy from Section 3. We now state the main security theorem for fp-GA-ODXT.

Theorem 1. *Let F and F_G be two independent secure PRFs, and let fow be a one-way trapdoor function. Then, our proposed construction fp-GA-ODXT is adaptively secure, forward private, and C2^- backward private for the leakage function $\mathcal{L}_{\text{fp-GA-ODXT}}$:*

$$\begin{aligned}
& |\Pr[\text{fp-GA-ODXT}_{\mathcal{A}}(\lambda) = 1] - \Pr[\text{Ideal}_{\mathcal{A}, \mathcal{S}}(\lambda) = 1]| \\
& \leq 4 \cdot \text{Adv}_{F_G, \mathcal{B}_1}^{\text{PRF}}(\lambda) + 2 \cdot \text{Adv}_{F, \mathcal{B}_2}^{\text{PRF}}(\lambda) + \text{Adv}_{\mathcal{B}_3}^{\text{GA-DDH}}(\lambda) + \frac{6\text{poly}(\lambda)}{2^{\lambda-1}}.
\end{aligned}$$

Proof. We will prove the theorem using the popular game hopping technique. In the following, we define the sequence of games between the challenger $\mathcal{C}$ and the adversary $\mathcal{A}$. The first game (i.e., Game $\mathcal{G}_0$) is identical to the real experiment $\text{fp-GA-ODXT}(\lambda, \mathcal{Q})$, and the distribution of the final game (i.e., Game $\mathcal{G}_5$) is such that a simulator $\mathcal{S}$ can simulate the transcripts only on input λ and the leakage function of fp-GA-ODXT without any knowledge of DB or $\mathcal{Q}$. Moreover, the games are so designed that the views of the adversary $\mathcal{A}$ in each pair of consecutive games are computationally indistinguishable.

Game $\mathcal{G}_0$. This game is identical to the real game i.e., $\text{fp-GA-ODXT}(\lambda, \mathcal{Q})$ experiment with a slight modification. Remember, the `stoken` values are computed and shared with the server so that the server itself can produce all the `xtoken` _{i,c} values for each $i \in \{2, 3, \dots, \mu\}$ and $c \in [c_{w_1}]$, from the only `xtoken` _{i,c_{w_i}} value it receives during a search query of the form $(w_1 \wedge w_2 \wedge \dots \wedge w_\mu)$. In this game, the challenger makes use of this fact, and it precomputes all the `xtoken` values on its own and sends them to the server during a search in place of sending only one `xtoken` and $(c_{w_1} - 1)$ `stokens`. The other values in the transcript corresponding to each update and conjunctive search query are generated identically as fp-GA-ODXT .

Game $\mathcal{G}_1$. This game is identical to Game $\mathcal{G}_0$ except that when generating the transcript for each update and conjunctive search queries we replace the PRF evaluations $F(K_T, \cdot)$, $F_G(K_X, \cdot)$, $F_G(K_Y, \cdot)$, and $F_G(K_Z, \cdot)$ with random functions $f_T(\cdot)$, $f_X(\cdot)$, $f_Y(\cdot)$, and $f_Z(\cdot)$, respectively. Note that the function f_T is uniformly sampled from the set of all functions that map λ -bit strings onto λ -bit strings. Likewise, the functions f_X , f_Y , and f_Z are uniformly sampled from the set of all functions that map λ -bit strings onto elements of group G . Since both F and F_G are secure PRFs, it is easy to see that

$$|\Pr[\mathcal{G}_1 = 1] - \Pr[\mathcal{G}_0 = 1]| \leq 4 \cdot \text{Adv}_{F_G, \mathcal{B}_1}^{\text{PRF}}(\lambda) + 2 \cdot \text{Adv}_{F, \mathcal{B}_2}^{\text{PRF}}(\lambda) + \frac{6 \cdot \text{poly}(\lambda)}{2^{\lambda-1}}. \quad (1)$$

Game $\mathcal{G}_2$. In the next step we change the way of generating `xtoken` values for each conjunctive search query $(t, q = w_1 \wedge w_2 \wedge \dots \wedge w_\mu)$. We have assumed that the corresponding `stern` is w_1 and w_i is involved in c_{w_i} many updates till timestamp t for all $i \in [\mu]$. First of all, we retrieve all the update queries involving w_1 as $\{(*, \text{op}_{1,c}, (w_1, \text{id}_{1,c}))\}_{c=1}^{c_{w_1}}$. The corresponding $\alpha_{1,c}$ values are computed as in the real fp-GA-ODXT scheme. Now, for each update $(*, \text{op}_{1,c}, (w_1, \text{id}_{1,c}))$ of w_1 and for each `xterm` w_i , we check if there is an update query of the form $(*, \text{op}_{1,c}, (w_i, \text{id}_{1,c}))$ in the query history $\mathcal{Q}$. If so, suppose that is the k^{th} query of w_i . Now, we sample `xtoken` _{i,j} as:

$$\text{xtoken}_{i,j} = f_u^{-1}(\text{xtag}_{i,k}) \cdot f_{\text{ow}}(w_i, k)^{-1} \cdot \alpha_1^{c-1}.$$

After all such `xtoken` values are generated, we just sample the remaining `xtoken` values uniformly at random from U . Thus, the distributions of `xtoken` in Game $\mathcal{G}_2$ and Game $\mathcal{G}_1$ are statistically indistinguishable. Hence,

$$|\Pr[\mathcal{G}_2 = 1] - \Pr[\mathcal{G}_1 = 1]| = 0. \quad (2)$$

Game $\mathcal{G}_3$. Moving forward, we now change the way we computed the α value in the previous game. We randomly sample α from the group G . Note that, f_Z is uniform random functions sampled from $\mathcal{Func}(\{0, 1\}^\lambda, G)$. Also, during any update operation, f_Z is evaluated exactly once for each input. Similarly, f_Y is sampled uniformly from the set of all random functions $\mathcal{Func}(\{0, 1\}^\lambda, G)$. So, the distributions of α being the product of an evaluation of f_Y with the inverse evaluation of f_Z , in Game $\mathcal{G}_2$ and Game $\mathcal{G}_3$ are statistically indistinguishable. Hence,

$$|\Pr[\mathcal{G}_3 = 1] - \Pr[\mathcal{G}_2 = 1]| = 0. \quad (3)$$

Game $\mathcal{G}_4$. Next, we modify the generation of the **xtag** values for the update queries. To be specific, we sample γ randomly from G and set **xtag** = $\gamma \star u$. Note that in Game $\mathcal{G}_3$, for an update query of the form $(*, op, (w, id))$ the **xtag** value is generated as $f_X(w) \cdot f_Y(id || op) \cdot f_{ow}(w, c)$, assuming this is the c^{th} update involving w . So, to distinguish the **xtag** values of Game $\mathcal{G}_4$ from that of Game $\mathcal{G}_3$, $\mathcal{A}$ has to distinguish $\gamma \star u$ from $(\alpha \cdot \beta) \star u$, where all of α, β, γ are random elements from G . Since we have assumed Decisional Diffie-Hellman (GA-DDH) assumption holds with respect to a group action $(G, U, \star)$, we have:

$$|\Pr[\mathcal{G}_4 = 1] - \Pr[\mathcal{G}_3 = 1]| \leq \mathbf{Adv}_{\mathcal{B}_3}^{\text{GA-DDH}}(\lambda). \quad (4)$$

Game $\mathcal{G}_5$. Finally, we modify the computation of **val** during the update queries: $f_T(w || c_w) \oplus id || op$ is replaced by evaluating $f_T(t)$, where t is the corresponding update timestamp. Note that each update query can be uniquely characterized by its corresponding global timestamp. Hence,

$$|\Pr[\mathcal{G}_5 = 1] - \Pr[\mathcal{G}_4 = 1]| = 0. \quad (5)$$

Now, we claim that it is possible to construct a simulator $\mathcal{S}$ that generates the transcript for each update and search query. Note that the transcripts will have to follow the same distribution as game $\mathcal{G}_5$. Also, remember that $\mathcal{S}$ does not have access to the actual queries, but it has access to the leakage functions $\mathcal{L}_{\text{fp-GA-ODXT}}$. The following Algorithm 5 and Algorithm 6 respectively capture the simulator's behavior during any update and conjunctive search query.

The theorem follows as we combine Eqn. 1, 2, 3, 4, 5.

5 Concrete Instantiations of fp-GA-ODXT

This section provides two concrete instantiations of fp-GA-ODXT. The first, classical instantiation is a close variant of ODXT that makes minimal changes

Algorithm 5 : $\mathcal{S}.Update(\perp)$

```
1:  $\text{addr}[t] \leftarrow f_T(t||0)$ 
2:  $\text{val}[t] \leftarrow f_T(t||1) \oplus \{0\}^\lambda$ 
3:  $\alpha[t], \beta \xleftarrow{\$} G$ 
4:  $\text{xtag}[t] \leftarrow \beta \star u$ 
5: Send  $\left( \{\text{addr}[t], \text{val}[t], \alpha[t]\}_{i=1}^{c_{w_1}} \right)$  to Server
```

Algorithm 6 : $\mathcal{S}.Search(\text{sp}^-(q), \text{TimeDB}(q), \text{Updates}^-(q))$

```
1: Identify  $t_{min}$  as minimum of  $\text{sp}^-(q)$ 
2: if  $t_{min} = t$  then
3:   Identify  $sterm$  as  $\overline{w_1}$ .
4:    $\text{Tab}_S[t] \leftarrow \overline{w_1}$ 
5: else
6:   Retrieve  $\overline{w_1}$  from  $\text{Tab}_S[t_{min}]$ 
7:   for  $i = 2, 3, \dots, \mu$  do
8:     Parse  $xterm$  as  $\delta_i$ 
9:      $r_i \xleftarrow{\$} \{0, 1\}^\lambda$ 
10:  $\text{Updates}(\overline{w_1}) \cup \left( \bigcup_{i=1}^{\mu} \text{Updates}(\overline{w_1}, \delta_i) \right) \leftarrow \text{UpdatesC}^-(q)$ 
11: if  $\text{Updates}(\overline{w_1}) = \perp$  then
12:   return  $\Phi$ 
13:  $\{t_{1,1}, t_{1,2}, \dots, t_{1,c_{w_1}}\} \leftarrow \text{Updates}(\overline{w_1})$ 
14: for  $c = 1, 2, \dots, c_{w_1}$  do
15:    $\text{saddr}_c \leftarrow f_T(t_{1,c}||0)$ 
16:   for  $i = 2, 3, \dots, \mu$  do
17:      $(t_{1,c}, t_{i,*}) \leftarrow \bigcup_{i=1}^{\mu} \text{Updates}(\overline{w_1}, \delta_i)$ 
18:     if  $(t_{1,c}, t_{i,*}) \neq \perp$  then
19:       Identify  $(t_{1,c}, t_{i,*})$  as  $(t_{1,c}, t_{i,k})$ 
20:        $\text{xtoken}_{i,c} \leftarrow f_u^{-1}(\text{xtag}[t_{i,c'}]) \cdot (\text{fow}(r_i, k))^{-1} \cdot (\alpha[t_{1,c}])^{-1}$ 
21:     else
22:        $\text{xtoken}_{i,c} \xleftarrow{\$} U$ 
23: Send  $\left( \{\text{saddr}_i\}_{i=1}^{c_{w_1}}, \{\{\text{xtoken}_{i,c}\}_{i=2}^n\}_{c=1}^{c_{w_1}}, \{\text{fow}(r_i, c_{\delta_i})\}_{i=2}^n \right)$  to Server
24:  $\text{Res} \leftarrow \text{TimeDB}(q)$ 
25: Output  $\text{Res}$ 
```

to ODXT to guarantee forward-privacy. Similar to ODXT, this instantiation of fp-GA-ODXT is based on the DDH assumption, with some additional techniques based on RSA groups. Next, we provide an instantiation of fp-GA-ODXT based on a combination of lattices and isogenies of elliptic curves that can be conjectured to be post-quantum secure. To our knowledge, this yields the first plausibly post-quantum conjunctive DSSE scheme.

5.1 Classical Instantiation from DDH and RSA

We choose a large prime p and work on $\mathbb{G}$, a cyclic group of order p where the DDH problem is assumed to be hard.

INSTANTIATION OF f_{ow} : We instantiate f_{ow} using a trapdoor one-way permutation Π and a PRF F as follows:

$$f_{ow}(w, i) := \Pi_{PK}^{(\ell-i)}(F_K(w)),$$

where ℓ is the parameter that denotes the maximum number of updates on any keyword. We use the RSA one-way trapdoor permutation, a popular choice for instantiating the trapdoor one-way permutation. Let p and q be two large primes of the same size and $N = pq$. We choose (e, d) such that $ed = 1 \pmod{\phi(N)}$, and define (N, e) to be the public key and (N, d) to be the trapdoor. Define the trapdoor one-way permutation as follows:

$$\Pi_{PK}(x) = x^e \pmod{N}, \quad \Pi_{SK}^{-1}(x) = x^d \pmod{N}.$$

Since, Π is a permutation, it holds that for all x , $\Pi_{PK}(x) = \Pi_{SK}^{-1}(x)$. Now it is easy to see that all three desired properties of f_{ow} hold:

- For each keyword, the client can efficiently compute $f_{ow}(w, i) := \Pi_{PK}^{(\ell-i)}(F_K(w))$.
- For each keyword w , the server can compute $f_{ow}(w, i-1)$ from the knowledge of $f_{ow}(w, i)$ by simply applying Π_{PK} :

$$f_{ow}(w, i-1) := \Pi_{PK}(f_{ow}(w, i)).$$

- For each keyword w , it is difficult for the server to efficiently compute $f_{ow}(w, i+1)$ from $f_{ow}(w, i)$, as computing $\Pi_{PK}^{(\ell-i-1)}(F_K(w))$ from $\Pi_{PK}^{(\ell-i)}(F_K(w))$ is essentially computing Π_{SK}^{-1} on $\Pi_{PK}^{(\ell-i-1)}(F_K(w))$, which is difficult.

Thus, the above instantiation satisfies all of the desired properties of f_{ow} , while relying on the hardness of the RSA one-way trapdoor permutations.

5.2 Post-Quantum Instantiation from Lattices and Isogenies

We present a second, post-quantum instantiation of our **fp-GA-ODXT** framework based on the (worst-case) hardness of lattice problems *and* the (average-case) hardness of computing isogeny maps between supersingular elliptic curves.

ISOGENY BASED INSTANTIATION OF EFFECTIVE GROUP ACTION: The group action layer of our framework **fp-GA-ODXT** can be instantiated using *any* effective group action instance satisfying the GA-DDH assumption 2. As noted in [2], the GA-DDH assumption plausibly holds over the group-action implied by isogenies of supersingular “oriented” elliptic curves over F_p . This group action underlies CSIDH [12] and several of its follow-ups, such as CSI-FiSh [3], Scallop [20], and PEGASIS [17].

The only known cryptanalytic studies of GA-DDH over any of these group action instances follow from cryptanalytic studies of CSIDH and its variants. Such studies suggest that using the same set of parameters as CSIDH-2048 would allow our GA-DDH-based instantiation of fp-GA-ODXT to achieve the same level of security as NIST’s level-1 security for post-quantum standards/candidates for key-encapsulation and digital signatures.

Remark. We remark here that the recent attacks on SIKE/SIDH [11,37,30] do not impact our proposed instantiation of fp-GA-ODXT. This is because SIKE and SIDH are based on an entirely different family of isogenies of supersingular “non-oriented” elliptic curves over F_{p^2} , which do not yield effective group actions. The aforementioned attacks on SIDH/SIKE *do not* impact the hardness of GA-DDH over isogenies of supersingular “oriented” elliptic curves over F_p , which CSIDH and its variants are based on.

SIS BASED INSTANTIATION OF f_{ow} : Let $m, n, N, \beta \in \mathbb{N}$ be parameters such that $\ell = \lceil \log_\beta N \rceil$, $m = n\ell$. If $N < \beta^\ell$, we write $N = \sum_{i \in [0, \ell-1]} N_i \cdot \beta^i$ in the β -ary expansion. If $N = \beta^\ell$, we write $N_0 = \dots = N_{\ell-2} = 0$ and $N_{\ell-1} = \beta$. For any matrix $\mathbf{A} \in \mathbb{Z}_N^{n \times m}$, we define the function $\text{SIS}_{\mathbf{A}, \beta} : \mathbb{Z}_N^n \rightarrow \mathbb{Z}_\beta^m$ that takes as input a vector $\mathbf{v} \in \mathbb{Z}_N^n$ and outputs a vector $\mathbf{z} \in \mathbb{Z}_\beta^m$ such that

$$\mathbf{A}\mathbf{z} = \mathbf{v} \pmod{N}.$$

Now recall from Section 4.3 and Theorem 1 that for correctness and security of our framework, we require the following properties from f_{ow} and f_{ow}^{-1} :

- (a) f_{ow} has the same input and output space,
- (b) f_{ow} is hard to compute publicly, but is efficiently computable given a trapdoor.
- (c) f_{ow}^{-1} is publicly and efficiently computable, and
- (d) f_{ow} and f_{ow}^{-1} are universal.

So, the natural starting point for a SIS-based realization of f_{ow} is the function $\text{SIS}_{\mathbf{A}, \beta}$ for an appropriate choice of parameters (m, n, N, β) for SIS hardness. The corresponding inverse function then becomes

$$\text{SIS}_{\mathbf{A}, \beta}^{-1}(\mathbf{z}) = \mathbf{A}\mathbf{z} \pmod{N}.$$

For this choice of f_{ow} , property (b) follows from SIS-hardness and the availability of SIS trapdoors, (c) follows from inspection, and (d) holds by the leftover hash lemma [24]. But the problem remains with property (a). For this purpose, we define the generalized gadget vector $\mathbf{g}$ and the generalized gadget matrix $\mathbf{G}$ as follows:

$$\mathbf{g}^T = (1 \quad \beta \quad \dots \quad \beta^\ell), \quad \mathbf{G} = \mathbf{I}_n \otimes \mathbf{g}^T,$$

where $\mathbf{I}_n$ is the identity matrix of dimension n and $\otimes$ denotes the matrix-vector tensor operator. We additionally define the operator $\mathbf{G}^{-1} : \mathbb{Z}_N^n \rightarrow \mathbb{Z}_\beta^m$ as the

β -ary expansion operator that maps any vector $\mathbf{v} \in \mathbb{Z}_N^n$ to the concatenation of the β -ary representation of its components, i.e.

$$\mathbf{G}^{-1}(\mathbf{v} = (v_1, \dots, v_n)) = ((v_{1,i})_{i \in [0, \ell-1]}, \dots, (v_{n,i})_{i \in [0, \ell-1]}),$$

such that for each $j \in [1, n]$, $v_j = \sum_{i \in [0, \ell-1]} v_{j,i} \cdot \beta^i$. Note that $\mathbf{G}^{-1}$ is efficiently computable. Now, we define the functions $h_{\mathbf{A}, \beta} : \mathbb{Z}_N^n \rightarrow \mathbb{Z}_N^n$ and $h_{\mathbf{A}, \beta}^{-1} : \mathbb{Z}_N^n \rightarrow \mathbb{Z}_N^n$ as follows:

$$h_{\mathbf{A}, \beta} = \mathbf{G} \cdot \text{SIS}_{\mathbf{A}, \beta}, \quad h_{\mathbf{A}, \beta}^{-1} = \mathbf{A} \mathbf{G}^{-1}.$$

With these definitions in hand, we observe that:

- For any $\mathbf{v} \in \mathbb{Z}_N^n$, we have

$$h_{\mathbf{A}, \beta}^{-1}(h_{\mathbf{A}, \beta}(\mathbf{v})) = \mathbf{v}.$$

- For any $\mathbf{v} \in \mathbb{Z}_N^n$, $h_{\mathbf{A}, \beta}^{-1}(\mathbf{v})$ is publicly and efficiently computable.
- Given only $\mathbf{A}$, computing $h_{\mathbf{A}, \beta}(\mathbf{v})$ is computationally hard assuming the hardness of the SIS problem for parameters (m, n, N, β) .
- Given a SIS trapdoor for $\mathbf{A}$ [31], one can efficiently compute $h_{\mathbf{A}, \beta}(\mathbf{v})$ for any (pseudo)random $\mathbf{v} \in \mathbb{Z}_N^n$ (by the leftover hash lemma [24]).

Now we will describe our concrete SIS based instantiation of $\mathbf{f}_{\text{ow}}$. For this purpose, let us first $F : \mathcal{K} \times \{0, 1\}^* \rightarrow \mathbb{Z}_N^n$ be a PRF family, and let $K \in \mathcal{K}$ be a PRF key. We note that such a PRF can be easily instantiated in practice. Now, for any keyword w , we set $\mathbf{f}_{\text{ow}}(w, 0) = F(K, w)$ and for each $i \geq 1$, we set

$$\mathbf{f}_{\text{ow}}(w, i+1) = \mathbf{G}(\text{SIS}_{\mathbf{A}, \beta}(\mathbf{v})) = h_{\mathbf{A}, \beta}(\mathbf{f}_{\text{ow}}(w, i)).$$

With this definition of $\mathbf{f}_{\text{ow}}$, property (a) follows immediately. Moreover, since $\mathbf{G}$ and $\mathbf{G}^{-1}$ are both bijective operators and efficiently computable, the properties (b), (c), and (d) remain unaffected.

Formally, we have the following:

- Given a SIS trapdoor for $\mathbf{A}$ (stored privately by the client), the client can compute $\mathbf{f}_{\text{ow}}(w, i+1) = h_{\mathbf{A}, \beta}(\mathbf{f}_{\text{ow}}(w, i))$ for any keyword w .
- The server can compute $\mathbf{f}_{\text{ow}}(w, i) = h_{\mathbf{A}, \beta}^{-1}(\mathbf{f}_{\text{ow}}(w, i+1))$ publicly and efficiently.
- Given only $\mathbf{A}$, it is computationally hard for the server to compute $\mathbf{f}_{\text{ow}}(w, i+1)$ given $\mathbf{f}_{\text{ow}}(w, i)$.

Thus, the above instantiation satisfies all of the desired properties of $\mathbf{f}_{\text{ow}}$, while relying on the post-quantum hardness of the SIS problem. Putting this together with known isogeny-based instantiations of GA-DDH-hard effective group actions [12, 3, 20] yields a post-quantum instantiation of fp-GA-ODXT.

PARAMETER CHOICES: The above construction can be instantiated in practice using any state-of-the-art implementation of isogeny-based group actions, such as CSIDH-512/1024 [12], or variants of CSIDH such as CSI-FiSh [3] and Scallop [20]. For the lattice-based implementation of f_{ow} , one would typically set parameters $n = 1024$, a suitable prime q (e.g., $q = 3329$ as in ML-KEM [4,32]), $m > 3n \log q$ for the leftover hash lemma [24] to hold, and β is set to an arbitrary (small) constant. The parameters n , N , and β (where N is the same as the prime q) are determined by standard choices for SIS-hardness [1,34,35,28,29]. The parameter m needs to simultaneously satisfy the hardness of SIS and the leftover hash lemma [24]. From [36], the above-mentioned condition on m satisfies both requirements simultaneously.

INTERACTION BETWEEN GROUP ACTIONS AND SIS-BASED OWF: In Algorithms 3 and 4, the interfacing between the effective group action and f_{ow} requires that the output space of f_{ow} is the same as the group $\mathcal{G}$. Note that, for an instantiation of effective group action based on CSIDH-2048, the group $\mathcal{G}$ is isomorphic to $\mathbb{Z}_M$ for a 1024-bit prime M . On the other hand, the input and output spaces of the SIS-based instantiation of f_{ow} are $\mathbb{Z}_N^n$, where n is the dimension parameter for LWE and N is the underlying prime. For $N = 3329$ and $n = 1024$ (which gives us the desired SIS hardness), $\mathbb{Z}_N^n$ is (almost) isomorphic to $\mathbb{Z}_L$ for 1035-bit L . For aligning these two spaces, we need a universal and collision-resistant encoding map from $\mathbb{Z}_M$ to $\mathbb{Z}_L$. This can be instantiated in practice using, say, SHA-512 (modeled as a universal and collision-resistant hash function). Since Algorithms 3 and 4 never use the reverse decoding from the group $\mathcal{G}$ to the output space of f_{ow} , we do not actually require a corresponding decoding function.

6 Implementation and Performance Evaluation

We implement the classical instantiation of the fp-GA-ODXT based on DDH and RSA from Section 5.1, and compare its performance with state-of-the-art conjunctive DSSE schemes: ODXT, SDSSE-CQ, and SDSSE-CQ-S. All code was written in C. Measurements were obtained by running this code on a *Dell Inc. Latitude 3420* machine equipped with an 11th Gen Intel® Core™ i7-1165G7 @ 2.80 GHz processor, running Ubuntu 22.04.3 LTS. The machine has 16 GB of RAM and 8 CPU cores. For cryptographic operations, we used the *OpenSSL* library, and group operations are implemented using the *secp256k1* elliptic curve. The PRFs are instantiated with *AES-128* in counter mode, while hash functions are instantiated using *SHA-256*. Our implementation utilized a single computer, with both client and server programs running on that machine.

We compare the performance of our proposed framework fp-GA-ODXT with ODXT, SDSSE-CQ, and SDSSE-CQ-S across two key aspects: (i) average update time and communication cost, and (ii) average search time and communication cost. We choose a synthetic dataset with 1 *million* documents. To ensure reliable results, we performed each operation 100 times and reported the average to

Table 2. Client-side update communication cost for each keyword-identifier pair (in bytes)

Operation Type	fp-GA-ODXT	SDSSE-CQ	SDSSE-CQ-S	ODXT
Addition	192	2 Aura update <i>id</i> size 116 & 64	2 Aura update <i>id</i> size 116 & 68	192
Deletion	192	0	0	192

eliminate any outlier effects. To ensure a fair comparison, we follow an experimental setup similar to that used in [47]. In the update phase, each document has been considered at least once with no faulty updates⁸. While calculating the average search time, search queries were performed on two conjunctions only, as queries with more keywords can be parallelized. In the following, we provide a detailed performance analysis of our proposed constructions and justify the implementation results.

6.1 Average Update Time and Communication Cost

In our proposed framework fp-GA-ODXT, the client sends the following values to the server during each update: ($\text{addr}, \text{val}, \alpha, \text{xtag}$). In our prototype implementation, sizes of these elements are *16 bytes*, *16 bytes*, *96 bytes*, and *64 bytes* respectively. Therefore, a total communication overhead of *192 bytes* per update is required for fp-GA-ODXT. Due to similar design choices, this is equivalent to the ODXT protocol of [33]. On the other hand, from [47] we can observe that the client-side update communication cost of SDSSE-CQ and SDSSE-CQ-S largely depends on the corresponding operation type and the underlying single keyword SSE scheme. For our experiments, we have considered Aura as the underlying SSE, following the publicly available implementation of these schemes in <https://github.com/MonashCybersecurityLab/SDSSE/tree/v1.0.0-paper>. For updates where $op = \text{add}$, SDSSE-CQ performs two updates in the underlying SSE scheme Aura, using update identifiers of sizes *116 bytes* and *64 bytes*, respectively. SDSSE-CQ-S does the same, except the second identifier is slightly larger, with a size of *68 bytes*. Following the construction of Aura [40], it is clear that the resulting update communication cost for both SDSSE-CQ and SDSSE-CQ-S will be much larger than our proposed scheme and ODXT as well. But, due to their design choices, SDSSE-CQ and SDSSE-CQ-S have a communication cost of *0 bytes* for updates where $op = \text{del}$. But this comes with a much larger computational overhead, which we will discuss in the subsequent sections. To conclude, unlike ODXT, our scheme ensures both forward privacy and stronger backward privacy, with a similar client-side update communication overhead compared to it. For clarity, these values are summarized in Table 2.

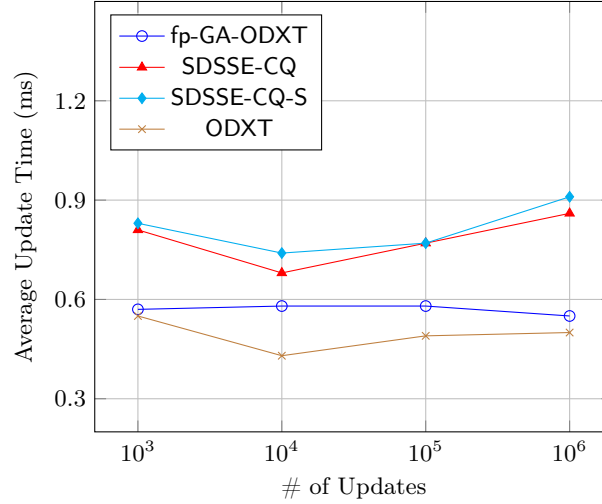
Next, we report the average update time for all these schemes. Since all the computations during updates are performed on the client side, we report the

⁸ faulty updates mean adding a keyword to a file which already contains it or deleting a keyword from a file which does not contain it

Table 3. Client-side update time per keyword-identifier pair (in milliseconds)

# of Updates	fp-GA-ODXT	SDSSE-CQ	SDSSE-CQ-S	ODXT
1000	0.57	0.81	0.83	0.55
10000	0.58	0.68	0.74	0.43
100000	0.58	0.77	0.77	0.49
1000000	0.55	0.86	0.91	0.50

time taken on the client side only. Our scheme exhibits performance very similar to ODXT, as expected. In contrast, SDSSE-CQ and SDSSE-CQ-S incur higher update times. This is because both these schemes are based on an advanced cryptographic primitive, dubbed Compressed Symmetric Revocable Encryption (CSRE) and need to generate an CSRE ciphertext during additions [47]. The results are summarized in Table 3. For ease of understanding, we also provide a corresponding graphical representation in Figure 1.

**Fig. 1.** Graph of Average Update Time

6.2 Average Search Time and Communication Cost

Next, we will evaluate the performance of our scheme in the conjunctive search queries. Let us first consider the average communication cost during such search queries. Recall that search queries are performed in three stages: the client generates the search tokens and sends them to the server, the server calculates the encrypted search result and sends it back to the client, and finally, the client decrypts the encrypted result set and outputs the final result. Clearly, the communication overhead for the latter two stages will be almost equivalent for all

Table 4. Client-side search communication cost (in megabytes)

$ \text{DB}(w_1) = 10^3$	fp-GA-ODXT	SDSSE-CQ	SDSSE-CQ-S	ODXT
$w_1 \wedge w_2$	0.112	0.104	0.20	0.080
$w_1 \wedge w_2 \wedge w_3$	0.112	0.188	0.38	0.144
$w_1 \wedge w_2 \wedge w_3 \wedge w_4$	0.112	0.272	0.56	0.208

Table 5. Client-side average search time (in milliseconds) for fixed $|\text{DB}(w_1)| = 10$

$ \text{DB}(w_2) $	fp-GA-ODXT	SDSSE-CQ	SDSSE-CQ-S	ODXT
10^1	1.525	0.227	0.277	1.548
10^2	1.552	0.207	0.637	1.532
10^3	1.425	0.206	3.763	1.489
10^4	1.52	0.208	32.111	1.541

the schemes. So we will only report the communication overhead for the client to send the search tokens to the server. As mentioned earlier, our scheme gains efficiency due to the novel technique of computing the *xtoken* on the fly at the server end. It is an intentional design choice for offloading the computational burden from the client to the server, as the server is generally equipped with more powerful hardware resources. The same is reflected in the comparison table as presented in Table 4. Note that we have evaluated the communication cost under three settings by fixing the frequency of the *sterm* to 10^3 and varying the number of *xterm* in the conjunction from 1 to 3. We observe that for two conjunctions, i.e., only one *xterm*, our scheme exhibits a client-side communication cost comparable to that of SDSSE-CQ and SDSSE-CQ-S, but slightly higher than that of ODXT. This is because in our protocol, the client needs to send an additional element of the multiplicative group, along with the search token, as compared to ODXT. However, the advantage of our scheme becomes evident as the number of *xterm* increases. In our approach, only one additional *xtoken* is required for each new *xterm* in the conjunction than before. In contrast, ODXT must send c_{w_1} many new *xtoken* for every new *xterm*, resulting in significantly higher communication overhead. For a similar reason, in SDSSE-CQ and SDSSE-CQ-S, the communication overhead increases linearly as the number of *xterm* increases in conjunction. For ease of understanding, we also provide a corresponding graphical representation in Figure 2.

Next, we will consider the average search time. As mentioned earlier, our novelty lies in offloading the computational burden from the client to the server, as the server is typically equipped with more powerful hardware resources. While reporting the average search time, we have only considered the time required for generating the search tokens on the client-side. For the other client computation, i.e., decrypting encrypted search results, the client takes almost the same time across all schemes.

We have reported average search times for two distinct query types, which are also quite familiar in the literature of conjunctive DSSE [33,47]. In the first query type, the frequency of *sterm* is fixed at 10, and the frequency of *xterm* is set to 10, 100, 1000, and 10000. We present the experimental results for the first query type in Table 5 and its graphical representation in Figure 3.

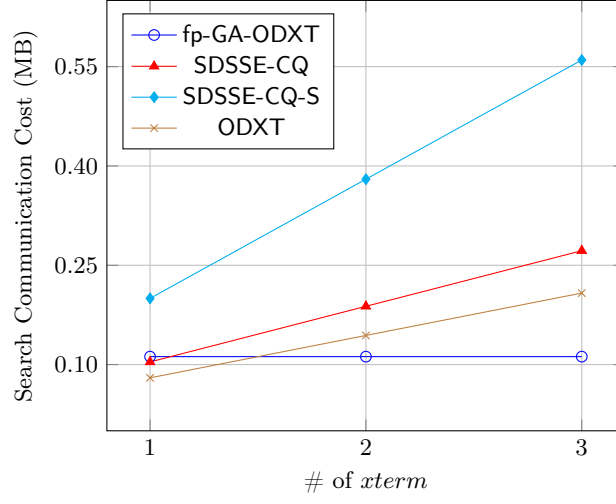


Fig. 2. Graph of Search Communication Cost

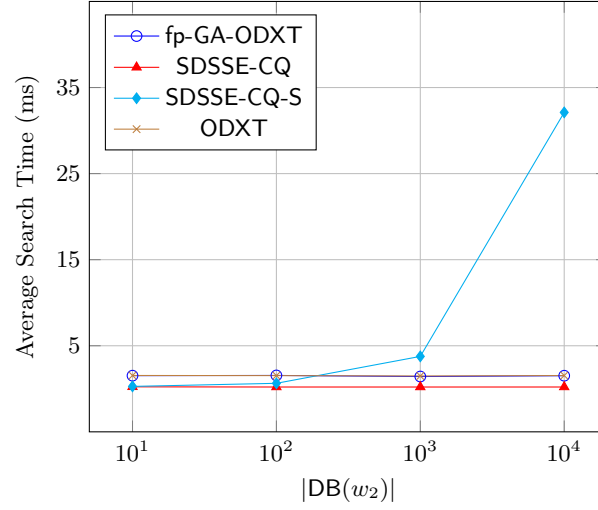


Fig. 3. Graph of Average Search Time for Fixed $|DB(w_1)| = 10$.

In the second query type, the frequency of $xterm$ is fixed at 10000, and the frequency of $stern$ is set to 10, 100, 1000, and 10000. For the second query type, the results are shown in Table 6, with the corresponding graphical representation in Figure 4.

It is clearly visible that our proposed scheme takes the minimal time to generate the search tokens. As discussed earlier, this happens because of our novel technique of offloading computation from the client to the resource-heavy

Table 6. Client-side average search time (in milliseconds) for fixed $|\text{DB}(w_2)| = 10000$

$ \text{DB}(w_1) $	fp-GA-ODXT	SDSSE-CQ	SDSSE-CQ-S	ODXT
10^1	1.52	0.208	32.111	1.541
10^2	5.151	2.067	38.724	5.14
10^3	14.481	21.213	58.515	15.219
10^4	108.724	184.809	261.458	115.129

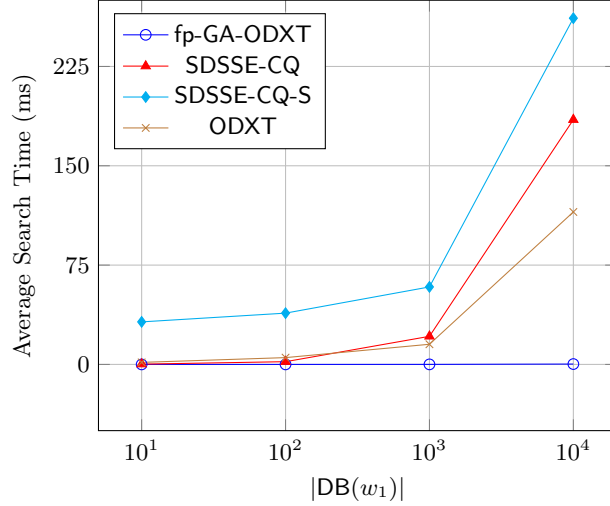


Fig. 4. Graph of Average Search Time for Fixed $|\text{DB}(w_2)| = 10000$.

server. In case of SDSSE-CQ and SDSSE-CQ-S, their search has two phases of token generation, one of which exactly coincides with ODXT. In the other part, it has to generate additional search tokens for each keyword in conjunction with the underlying single keyword scheme *Aura*. This makes their search equivalent to a naive single-keyword search for each keyword in the conjunction, resulting in larger client-side search time than ODXT and hence our schemes. To sum up, in both settings, we get an edge over all other schemes, which supports the claim of efficiency of our proposed framework fp-GA-ODXT.

7 Conclusion

In this paper, we addressed the open question of designing a practically efficient, low-leakage, full-fledged forward and backward private conjunctive DSSE scheme with small communication requirements and small client-side overheads. As a first contribution, we introduced the first systematic and rigorous formal notions of backward privacy for conjunctive queries, namely Type C1, C2, and C3, naturally extending the notion of Type-I, Type-II, and Type-III backward privacy for single-keyword DSSE schemes. We additionally defined a backward

privacy notion specific to the OXT framework, dubbed $C2^-$, which is similar to but a stronger notion than $C2$.

Next, as our main contribution, we proposed a new generic conjunctive DSSE framework called **fp-GA-ODXT** that improves upon both ODXT and SDSSE-CQ-variants by achieving full-fledged forward privacy and strong backward privacy guarantees, while incurring lower client-side computation and smaller communication overheads. We demonstrated and analyzed two instantiations of our fp-GA-ODXT framework:

- A concretely efficient, classically secure instance based on discrete log-hard groups and an RSA-based trapdoor permutation, which we implemented and benchmarked. We implement and benchmark this instantiation, and compare it with both ODXT and SDSSE-CQ/SDSSE-CQ-S [47]. Our experiments demonstrate that we achieve smaller communication overheads and client-side computation for both updates and conjunctive searches as compared to both ODXT and SDSSE-CQ-variants. These come at the cost of an increase in server computation, which we view as a reasonable trade-off, especially in practical scenarios where the SSE client is significantly resource-constrained.
- A post-quantum instantiation based on (worst-case) hardness of lattice problems *and* the (average-case) hardness of computing isogeny maps between supersingular elliptic curves. This solves the longstanding open question of designing asymptotically efficient, forward and backward private conjunctive DSSE schemes with post-quantum security.

We leave it as an interesting open question to extend the **fp-GA-ODXT** framework to support other query types such as substring/range queries, as well as to support multi-client scenarios.

References

1. Ajtai, M.: Generating hard instances of lattice problems (extended abstract). In: Miller, G.L. (ed.) Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996. pp. 99–108. ACM (1996). doi:10.1145/237814.237838, <https://doi.org/10.1145/237814.237838>
2. Alami, N., Feo, L.D., Montgomery, H., Patranabis, S.: Cryptographic group actions and applications. In: Moriai, S., Wang, H. (eds.) Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part II. pp. 411–439. Lecture Notes in Computer Science, Springer (2020). doi:10.1007/978-3-030-64834-3_14, https://doi.org/10.1007/978-3-030-64834-3_14
3. Beullens, W., Kleinjung, T., Vercauteren, F.: Csi-fish: Efficient isogeny based signatures through class group computations. In: Galbraith, S.D., Moriai, S. (eds.) Advances in Cryptology - ASIACRYPT 2019 - 25th International Conference on the Theory and Application of Cryptology and Information Security, Kobe, Japan, December 8-12, 2019, Proceedings, Part I. pp. 227–247. Lecture

- Notes in Computer Science, Springer (2019). doi:10.1007/978-3-030-34578-5_9, https://doi.org/10.1007/978-3-030-34578-5_9
4. Bos, J.W., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS - kyber: A cca-secure module-lattice-based KEM. In: 2018 IEEE European Symposium on Security and Privacy, EuroS&P 2018, London, United Kingdom, April 24-26, 2018. pp. 353–367. IEEE (2018). doi:10.1109/EUROSP.2018.00032, <https://doi.org/10.1109/EuroSP.2018.00032>
 5. Bost, R.: $\Sigma\phi\phi\phi$: Forward secure searchable encryption. In: Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds.) Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016. pp. 1143–1154. ACM (2016). doi:10.1145/2976749.2978303, <https://doi.org/10.1145/2976749.2978303>
 6. Bost, R., Minaud, B., Ohrimenko, O.: Forward and backward private searchable encryption from constrained cryptographic primitives. In: Thuraisingham, B., Evans, D., Malkin, T., Xu, D. (eds.) Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017. pp. 1465–1482. ACM (2017). doi:10.1145/3133956.3133980, <https://doi.org/10.1145/3133956.3133980>
 7. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: Fully homomorphic encryption without bootstrapping. Electron. Colloquium Comput. Complex. **TR11-111** (2011), <https://eccc.weizmann.ac.il/report/2011/111>
 8. Cash, D., Grubbs, P., Perry, J., Ristenpart, T.: Leakage-abuse attacks against searchable encryption. In: Ray, I., Li, N., Kruegel, C. (eds.) Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, October 12-16, 2015. pp. 668–679. ACM (2015). doi:10.1145/2810103.2813700, <https://doi.org/10.1145/2810103.2813700>
 9. Cash, D., Jaeger, J., Jarecki, S., Jutla, C.S., Krawczyk, H., Rosu, M., Steiner, M.: Dynamic searchable encryption in very-large databases: Data structures and implementation. In: 21st Annual Network and Distributed System Security Symposium, NDSS 2014, San Diego, California, USA, February 23-26, 2014. The Internet Society (2014)
 10. Cash, D., Jarecki, S., Jutla, C.S., Krawczyk, H., Rosu, M., Steiner, M.: Highly-scalable searchable symmetric encryption with support for boolean queries. In: Canetti, R., Garay, J.A. (eds.) Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part I. Lecture Notes in Computer Science, vol. 8042, pp. 353–373. Springer (2013). doi:10.1007/978-3-642-40041-4_20, https://doi.org/10.1007/978-3-642-40041-4_20
 11. Castryck, W., Decru, T.: An efficient key recovery attack on SIDH. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V. pp. 423–447. Lecture Notes in Computer Science, Springer (2023). doi:10.1007/978-3-031-30589-4_15, https://doi.org/10.1007/978-3-031-30589-4_15
 12. Castryck, W., Lange, T., Martindale, C., Panny, L., Renes, J.: CSIDH: an efficient post-quantum commutative group action. In: Peyrin, T., Galbraith, S.D. (eds.) Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part III. pp. 395–427. Lecture

- Notes in Computer Science, Springer (2018). doi:10.1007/978-3-030-03332-3_15, https://doi.org/10.1007/978-3-030-03332-3_15
13. Chamani, J.G., Papadopoulos, D., Papamanthou, C., Jalili, R.: New constructions for forward and backward private symmetric searchable encryption. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018. pp. 1038–1055. ACM (2018). doi:10.1145/3243734.3243833, <https://doi.org/10.1145/3243734.3243833>
 14. Chang, Y., Mitzenmacher, M.: Privacy preserving keyword searches on remote encrypted data. In: Ioannidis, J., Keromytis, A.D., Yung, M. (eds.) Applied Cryptography and Network Security, Third International Conference, ACNS 2005, New York, NY, USA, June 7-10, 2005, Proceedings. Lecture Notes in Computer Science, vol. 3531, pp. 442–455 (2005). doi:10.1007/11496137_30, https://doi.org/10.1007/11496137_30
 15. Chatterjee, S., Puria, S.K.P., Shah, A.: Efficient backward private searchable encryption. *J. Comput. Secur.* **28**(2), 229–267 (2020). doi:10.3233/JCS-191322, <https://doi.org/10.3233/JCS-191322>
 16. Curtmola, R., Garay, J.A., Kamara, S., Ostrovsky, R.: Searchable symmetric encryption: improved definitions and efficient constructions. In: Juels, A., Wright, R.N., di Vimercati, S.D.C. (eds.) Proceedings of the 13th ACM Conference on Computer and Communications Security, CCS 2006, Alexandria, VA, USA, October 30 - November 3, 2006. pp. 79–88. ACM (2006). doi:10.1145/1180405.1180417, <https://doi.org/10.1145/1180405.1180417>
 17. Dartois, P., Eriksen, J.K., Fouotsa, T.B., Merdy, A.H.L., Invernizzi, R., Robert, D., Rueger, R., Vercauteren, F., Wesolowski, B.: PEGASIS: practical effective class group action using 4-dimensional isogenies. In: Kalai, Y.T., Kamara, S.F. (eds.) Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part I. Lecture Notes in Computer Science, vol. 16000, pp. 67–99. Springer (2025). doi:10.1007/978-3-032-01855-7_3, https://doi.org/10.1007/978-3-032-01855-7_3
 18. Das, B.C.: Conjunctive dynamic SSE schemes under scrutiny: Exposing privacy issues in SDSSE-CQ-S and VCDSSSE. In: Nitaj, A., Petkova-Nikova, S., Rijmen, V. (eds.) Progress in Cryptology - AFRICACRYPT 2025 - 16th International Conference on Cryptology in Africa, Rabat, Morocco, July 21-23, 2025, Proceedings. Lecture Notes in Computer Science, vol. 15651, pp. 417–440. Springer (2025). doi:10.1007/978-3-031-97260-7_19, https://doi.org/10.1007/978-3-031-97260-7_19
 19. Faber, S., Jarecki, S., Krawczyk, H., Nguyen, Q., Rosu, M., Steiner, M.: Rich queries on encrypted data: Beyond exact matches. In: Pernul, G., Ryan, P.Y.A., Weippl, E.R. (eds.) Computer Security - ESORICS 2015 - 20th European Symposium on Research in Computer Security, Vienna, Austria, September 21-25, 2015, Proceedings, Part II. Lecture Notes in Computer Science, vol. 9327, pp. 123–145. Springer (2015). doi:10.1007/978-3-319-24177-7_7
 20. Feo, L.D., Fouotsa, T.B., Kutas, P., Leroux, A., Merz, S., Panny, L., Wesolowski, B.: SCALLOP: scaling the csi-fish. In: Boldyreva, A., Kolesnikov, V. (eds.) Public-Key Cryptography - PKC 2023 - 26th IACR International Conference on Practice and Theory of Public-Key Cryptography, Atlanta, GA, USA, May 7-10, 2023, Proceedings, Part I. pp. 345–375. Lecture Notes in Computer Science,

- Springer (2023). doi:10.1007/978-3-031-31368-4_13, https://doi.org/10.1007/978-3-031-31368-4_13
21. Goh, E.: Secure indexes. IACR Cryptol. ePrint Arch. p. 216 (2003), <http://eprint.iacr.org/2003/216>
 22. Goldreich, O., Ostrovsky, R.: Software protection and simulation on oblivious rams. J. ACM **43**(3), 431–473 (1996). doi:10.1145/233551.233553, <https://doi.org/10.1145/233551.233553>
 23. Hu, C., Song, X., Liu, P., Xin, Y., Xu, Y., Duan, Y., Hao, R.: Forward secure conjunctive-keyword searchable encryption. IEEE Access **7**, 35035–35048 (2019). doi:10.1109/ACCESS.2019.2902855, <https://doi.org/10.1109/ACCESS.2019.2902855>
 24. Impagliazzo, R., Levin, L.A., Luby, M.: Pseudo-random generation from one-way functions (extended abstracts). In: Johnson, D.S. (ed.) Proceedings of the 21st Annual ACM Symposium on Theory of Computing, May 14-17, 1989, Seattle, Washington, USA. pp. 12–24. ACM (1989). doi:10.1145/73007.73009, <https://doi.org/10.1145/73007.73009>
 25. Kamara, S., Moataz, T.: Boolean searchable symmetric encryption with worst-case sub-linear complexity. In: Coron, J., Nielsen, J.B. (eds.) Advances in Cryptology - EUROCRYPT 2017 - 36th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Paris, France, April 30 - May 4, 2017, Proceedings, Part III. Lecture Notes in Computer Science, vol. 10212, pp. 94–124 (2017). doi:10.1007/978-3-319-56617-7_4, https://doi.org/10.1007/978-3-319-56617-7_4
 26. Kamara, S., Papamanthou, C., Roeder, T.: Dynamic searchable symmetric encryption. In: Yu, T., Danezis, G., Gligor, V.D. (eds.) the ACM Conference on Computer and Communications Security, CCS'12, Raleigh, NC, USA, October 16-18, 2012. pp. 965–976. ACM (2012). doi:10.1145/2382196.2382298, <https://doi.org/10.1145/2382196.2382298>
 27. Lai, S., Patranabis, S., Sakzad, A., Liu, J.K., Mukhopadhyay, D., Steinfeld, R., Sun, S., Liu, D., Zuo, C.: Result pattern hiding searchable encryption for conjunctive queries. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018. pp. 745–762. ACM (2018). doi:10.1145/3243734.3243753, <https://doi.org/10.1145/3243734.3243753>
 28. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: Gilbert, H. (ed.) Advances in Cryptology - EUROCRYPT 2010, 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Monaco / French Riviera, May 30 - June 3, 2010. Proceedings. pp. 1–23. Lecture Notes in Computer Science, Springer (2010). doi:10.1007/978-3-642-13190-5_1, https://doi.org/10.1007/978-3-642-13190-5_1
 29. Lyubashevsky, V., Peikert, C., Regev, O.: A toolkit for ring-lwe cryptography. In: Johansson, T., Nguyen, P.Q. (eds.) Advances in Cryptology - EUROCRYPT 2013, 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Athens, Greece, May 26-30, 2013. Proceedings. pp. 35–54. Lecture Notes in Computer Science, Springer (2013). doi:10.1007/978-3-642-38348-9_3, https://doi.org/10.1007/978-3-642-38348-9_3
 30. Maino, L., Martindale, C., Panny, L., Pope, G., Wesolowski, B.: A direct key recovery attack on SIDH. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27,

- 2023, Proceedings, Part V. pp. 448–471. Lecture Notes in Computer Science, Springer (2023). doi:10.1007/978-3-031-30589-4_16, https://doi.org/10.1007/978-3-031-30589-4_16
31. Micciancio, D., Peikert, C.: Trapdoors for lattices: Simpler, tighter, faster, smaller. In: Pointcheval, D., Johansson, T. (eds.) *Advances in Cryptology - EUROCRYPT 2012 - 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Cambridge, UK, April 15–19, 2012. Proceedings. pp. 700–718. Lecture Notes in Computer Science, Springer (2012). doi:10.1007/978-3-642-29011-4_41, https://doi.org/10.1007/978-3-642-29011-4_41
 32. National Institute of Standards and Technology: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM). FIPS 203 (2024), available at <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf>
 33. Patranabis, S., Mukhopadhyay, D.: Forward and backward private conjunctive searchable symmetric encryption. In: 28th Annual Network and Distributed System Security Symposium, NDSS 2021, virtually, February 21–25, 2021. The Internet Society (2021), <https://www.ndss-symposium.org/ndss-paper/forward-and-backward-private-conjunctive-searchable-symmetric-encryption/>
 34. Regev, O.: New lattice-based cryptographic constructions. *J. ACM* **51**(6), 899–942 (2004). doi:10.1145/1039488.1039490, <https://doi.org/10.1145/1039488.1039490>
 35. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. *J. ACM* **56**(6), 34:1–34:40 (2009). doi:10.1145/1568318.1568324, <https://doi.org/10.1145/1568318.1568324>
 36. Regev, O.: The learning with errors problem (invited survey). In: *Proceedings of the 25th Annual IEEE Conference on Computational Complexity, CCC 2010*, Cambridge, Massachusetts, USA, June 9–12, 2010. pp. 191–204. IEEE Computer Society (2010). doi:10.1109/CCC.2010.26, <https://doi.org/10.1109/CCC.2010.26>
 37. Robert, D.: Breaking SIDH in polynomial time. In: Hazay, C., Stam, M. (eds.) *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Lyon, France, April 23–27, 2023, Proceedings, Part V. pp. 472–503. Lecture Notes in Computer Science, Springer (2023). doi:10.1007/978-3-031-30589-4_17, https://doi.org/10.1007/978-3-031-30589-4_17
 38. Song, D.X., Wagner, D.A., Perrig, A.: Practical techniques for searches on encrypted data. In: 2000 IEEE Symposium on Security and Privacy, Berkeley, California, USA, May 14–17, 2000. pp. 44–55. IEEE Computer Society (2000). doi:10.1109/SECPRI.2000.848445, <https://doi.org/10.1109/SECPRI.2000.848445>
 39. Stefanov, E., van Dijk, M., Shi, E., Fletcher, C.W., Ren, L., Yu, X., Devadas, S.: Path ORAM: an extremely simple oblivious RAM protocol. In: Sadeghi, A., Gligor, V.D., Yung, M. (eds.) 2013 ACM SIGSAC Conference on Computer and Communications Security, CCS’13, Berlin, Germany, November 4–8, 2013. pp. 299–310. ACM (2013). doi:10.1145/2508859.2516660, <https://doi.org/10.1145/2508859.2516660>
 40. Sun, S., Steinfeld, R., Lai, S., Yuan, X., Sakzad, A., Liu, J.K., Nepal, S., Gu, D.: Practical non-interactive searchable encryption with forward and backward privacy. In: 28th Annual Network and Distributed System Security Symposium, NDSS 2021, virtually, February 21–25, 2021. The Internet Society (2021)

41. Sun, S., Yuan, X., Liu, J.K., Steinfeld, R., Sakzad, A., Vo, V., Nepal, S.: Practical backward-secure searchable encryption from symmetric puncturable encryption. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018. pp. 763–780. ACM (2018). doi:10.1145/3243734.3243782, <https://doi.org/10.1145/3243734.3243782>
42. Talapatra, D., Patranabis, S., Mukhopadhyay, D.: Conjunctive searchable symmetric encryption from hard lattices. In: 8th IEEE European Symposium on Security and Privacy, EuroS&P 2023, Delft, Netherlands, July 3-7, 2023. pp. 958–978. IEEE (2023). doi:10.1109/EUROSP57164.2023.00061, <https://doi.org/10.1109/EuroSP57164.2023.00061>
43. Talapatra, D., Patranabis, S., Mukhopadhyay, D.: Highly scalable searchable symmetric encryption for boolean queries from NTRU lattice trapdoors. IACR Commun. Cryptol. **2**(2), 6 (2025). doi:10.62056/AE89N59P1, <https://doi.org/10.62056/ae89n59p1>
44. Wang, Y., Wang, J., Sun, S., Miao, M., Chen, X.: Toward forward secure SSE supporting conjunctive keyword search. IEEE Access **7**, 142762–142772 (2019). doi:10.1109/ACCESS.2019.2944246, <https://doi.org/10.1109/ACCESS.2019.2944246>
45. Wu, Z., Li, K.: Vbtree: forward secure conjunctive queries over encrypted data for cloud computing. VLDB J. **28**(1), 25–46 (2019). doi:10.1007/S00778-018-0517-6, <https://doi.org/10.1007/s00778-018-0517-6>
46. Zhang, Y., Katz, J., Papamanthou, C.: All your queries are belong to us: The power of file-injection attacks on searchable encryption. In: Holz, T., Savage, S. (eds.) 25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016. pp. 707–720. USENIX Association (2016), <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/zhang>
47. Zuo, C., Lai, S., Sun, S., Yuan, X., Liu, J.K., Shao, J., Wang, H., Zhu, L., Cui, S.: Searchable encryption for conjunctive queries with extended forward and backward privacy. Proc. Priv. Enhancing Technol. **2025**(1), 440–455 (2025). doi:10.56553/POPETS-2025-0024, <https://doi.org/10.56553/popets-2025-0024>
48. Zuo, C., Sun, S., Liu, J.K., Shao, J., Pieprzyk, J.: Dynamic searchable symmetric encryption schemes supporting range queries with forward (and backward) security. In: López, J., Zhou, J., Soriano, M. (eds.) Computer Security - 23rd European Symposium on Research in Computer Security, ESORICS 2018, Barcelona, Spain, September 3-7, 2018, Proceedings, Part II. Lecture Notes in Computer Science, vol. 11099, pp. 228–246. Springer (2018). doi:10.1007/978-3-319-98989-1_12, https://doi.org/10.1007/978-3-319-98989-1_12
49. Zuo, C., Sun, S., Liu, J.K., Shao, J., Pieprzyk, J., Wei, G.: Forward and backward private dynamic searchable symmetric encryption for conjunctive queries. IACR Cryptol. ePrint Arch. p. 1357 (2020), <https://eprint.iacr.org/2020/1357>